

# **Presentation 4**

## **Team 4**

Benjamin Pilnick, Zoe Lively, Akshay Anand, Isabella Toth

# OPTUNA Tuning

[OPTUNA Code](#)

[DataBase OPTUNA Results](#)

[Code to obtain results](#)

# Hyperparameter Value Ranges

| Parameter          | Round 1    | Round 2    | Round 3     | Round 4    |
|--------------------|------------|------------|-------------|------------|
| Learning Rate      | 3e-5, 1e-3 | 1e-5, 2e-4 | 2e-5, 8e-5  | 3e-5, 1e-4 |
| Weight Decay       | 1e-6, 2e-3 | 1e-6, 2e-3 | 5e-6, 1e-4  | 1e-5, 5e-4 |
| Patch Dropout      | 0.15, 0.55 | 0.35, 0.70 | 0.40, 0.70  | 0.45, 0.60 |
| Classifier Dropout | 0.15, 0.55 | 0.10, 0.40 | 0.10, 0.40  | 0.10, 0.40 |
| Class weight       | 2.0, 4.0   | 1.5, 3.0   | 1.5, 3.0    | 1.5, 3.0   |
| Entropy Lambda     | 1e-5, 1e-2 | 1e-5, 1e-2 | 1e-5, 1e-3, | 1e-5, 1e-3 |
| Sobol Startup      | 16         | 10         | 8           | 8          |
| TPE trials         | 48         | 54         | 56          | 16         |

# Best Hyperparameter Values

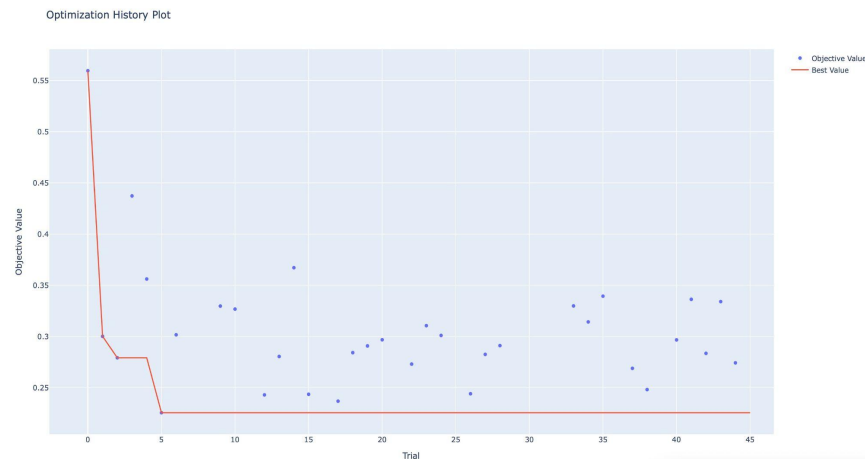
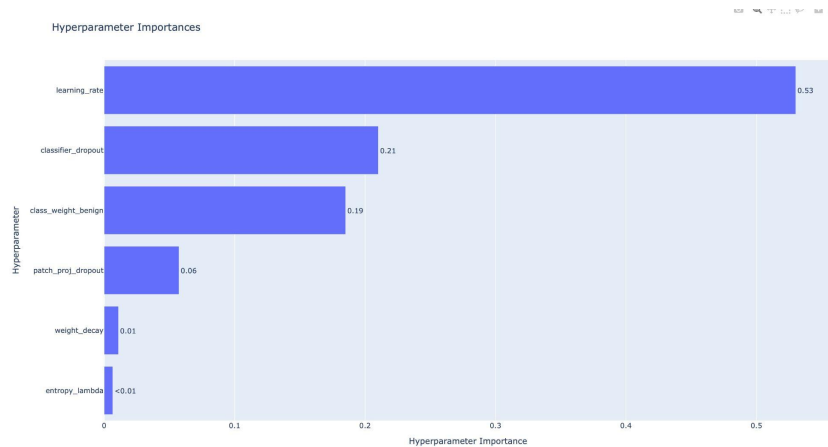
| Parameter          | Round 1 | Round 2 | Round 3 | Round 4 |
|--------------------|---------|---------|---------|---------|
| Learning Rate      | 5.23e-5 | 6.09e-5 | 4.62e-5 | N/A     |
| Weight Decay       | 4.70e-4 | 4.44e-5 | 2.23e-5 | N/A     |
| Patch Dropout      | 0.511   | 0.490   | 0.520   | N/A     |
| Classifier Dropout | 0.198   | 0.331   | 0.331   | N/A     |
| Class weight       | 2.137   | 2.887   | 2.887   | N/A     |
| Entropy Lambda     | 9.98e-4 | 4.78e-5 | 2.84e-5 | N/A     |
| Val Loss           | 0.2255  | 0.2575  | 0.2646  | N/A     |
| Best Epoch         | 9       | 6       | 6       | N/A     |

# Results Summary

| Round   | Accuracy | Loss    | Benign Recall | HighGrade Recall |
|---------|----------|---------|---------------|------------------|
| Round 1 | 0.827    | 0.589   | 0.769         | 0.859            |
| Round 2 | 0.838    | 0.672   | 0.769         | 0.8756           |
| Round 3 | 0.827    | 0.61223 | 0.769         | 0.86034          |
| Round 4 | N/A      | N/A     | N/A           | N/A              |

Best Model = Round 1. Lowest Validation and test loss with similar recall and accuracy to the other rounds.

# Best Model Param Importances/Optimization



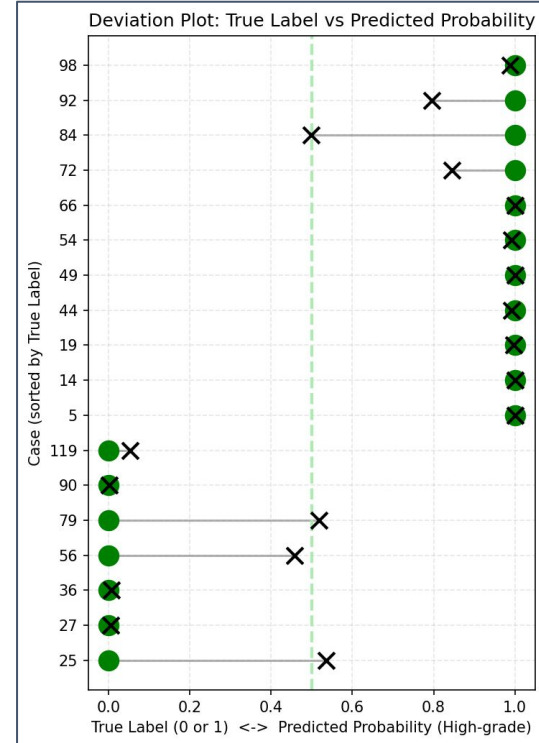
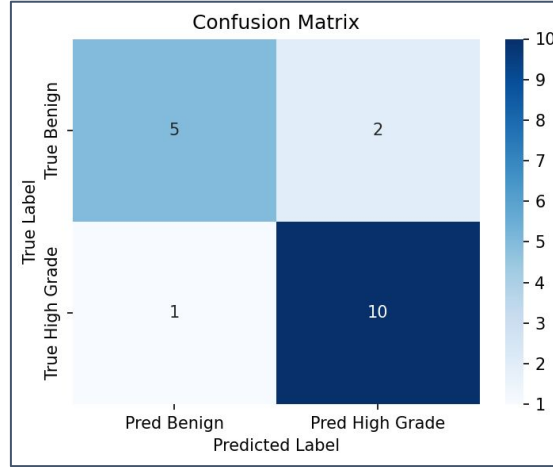
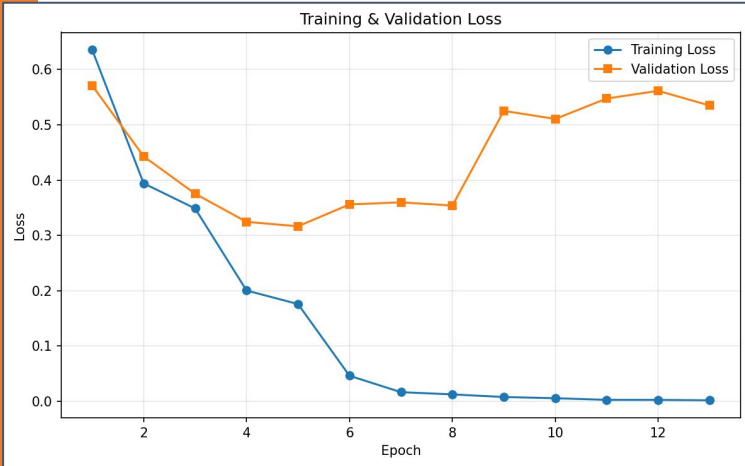
## Takeaways:

- Early Sobol trials optimized validation loss across rounds.
- Weight\_decay/entropy lambda that were high in round 1 had the least importance. But other param importances for Round 2 & 3 show higher importance.
  - Round 4 resolve whether weight\_decay/entropy lambda are important by widening the parameter combinations.

## Final Tuned Model Results: Round 1, Split 1

Isabella

Github:  
STAT390\_Team4/Presentation\_4\_cod  
e\_Bella/split1\_vis.ipynb



**\*\* For the tuned model, graphs/visualizations were created for each of the 5 splits of the round with the best hyperparameter configuration\*\***

- Training loss goes to 0, validation loss rises after epoch 5 = poor generalization/OVERFITTING PRESENT
- 2 false positives, 1 false negative
- Not confident misclassifications, resting near 0.5 threshold

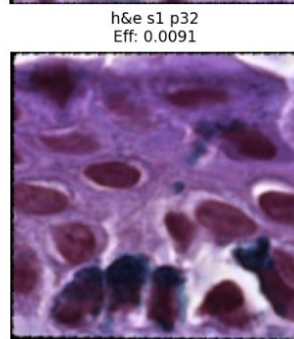
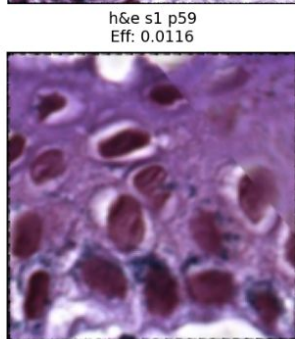
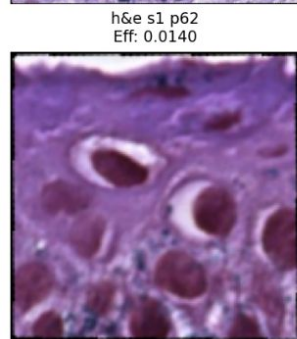
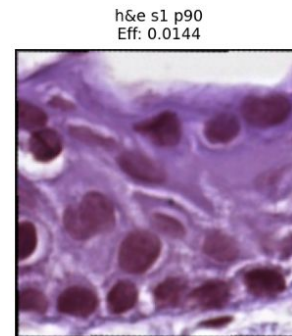
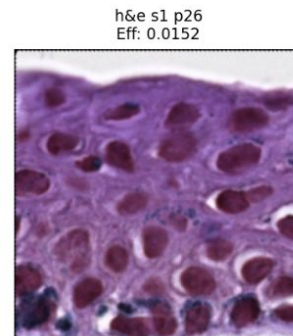
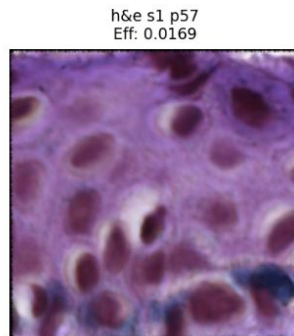
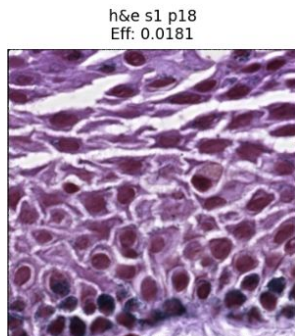
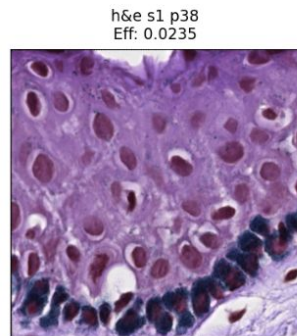
# Final Tuned Model Results: Round 1, Split 1

## Misclassified Benign- Case 25

Isabella

GitHub:  
run\_20260314\_071332/attention\_an  
alysis/case\_effective\_patches/case\_  
25\_top\_effective\_patches.png

**Case 25 - Top Effective Patches**



- Not confident misclassification
- No singular attention weight domination

# Final Tuned Model Results: Round 1, Split 1

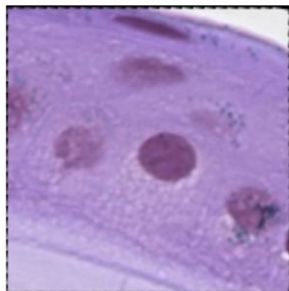
## Misclassified Benign- Case 79

Isabella

GitHub:  
run\_20260314\_071332/attention\_an  
alysis/case\_effective\_patches/case\_  
79\_top\_effective\_patches.png

**Case 79 - Top Effective Patches**

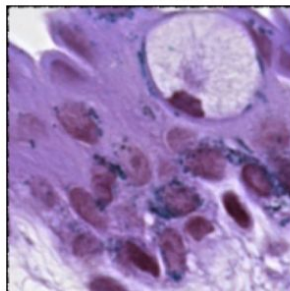
h&e s0 p115  
Eff: 0.0153



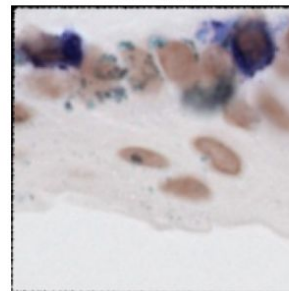
h&e s2 p33  
Eff: 0.0144



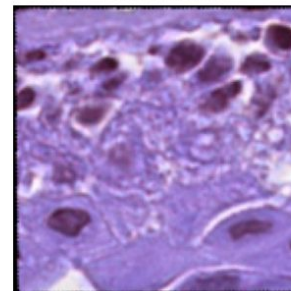
h&e s2 p58  
Eff: 0.0114



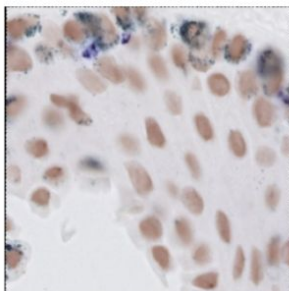
melan s0 p105  
Eff: 0.0101



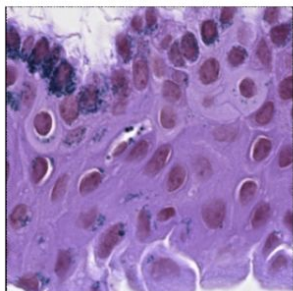
h&e s2 p34  
Eff: 0.0090



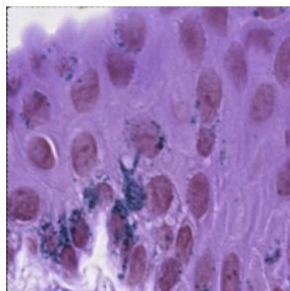
melan s0 p44  
Eff: 0.0089



h&e s2 p22  
Eff: 0.0079



h&e s2 p63  
Eff: 0.0071



- Not confident misclassification
- No singular attention weight domination

# Final Tuned Model Results: Round 1, Split 1

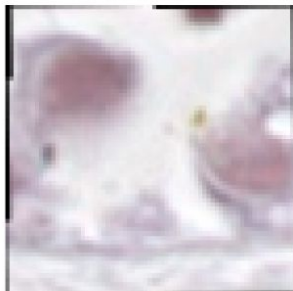
## Misclassified High Grade- Case 84

Isabella

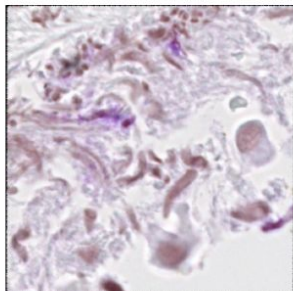
GitHub:  
run\_20260314\_071332/attention\_an  
alysis/case\_effective\_patches/case\_  
84\_top\_effective\_patches.png

### Case 84 - Top Effective Patches

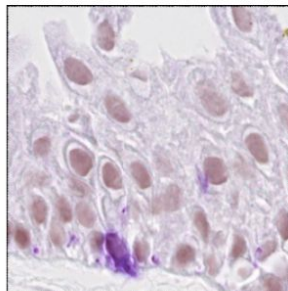
melan s0 p124  
Eff: 0.0277



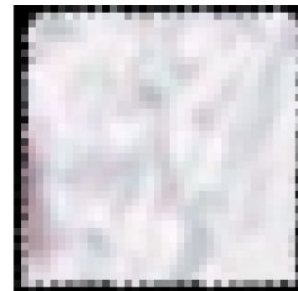
melan s0 p1  
Eff: 0.0259



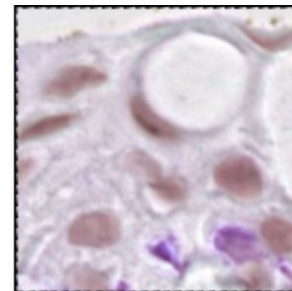
melan s0 p144  
Eff: 0.0250



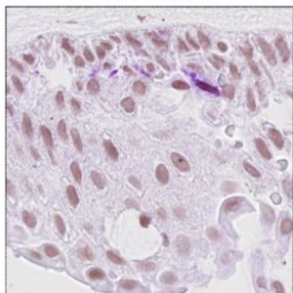
melan s0 p168  
Eff: 0.0232



melan s0 p155  
Eff: 0.0206



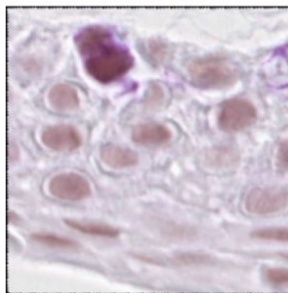
melan s0 p26  
Eff: 0.0194



melan s0 p143  
Eff: 0.0152



melan s0 p31  
Eff: 0.0148



- Not confident misclassification
- No singular attention weight domination

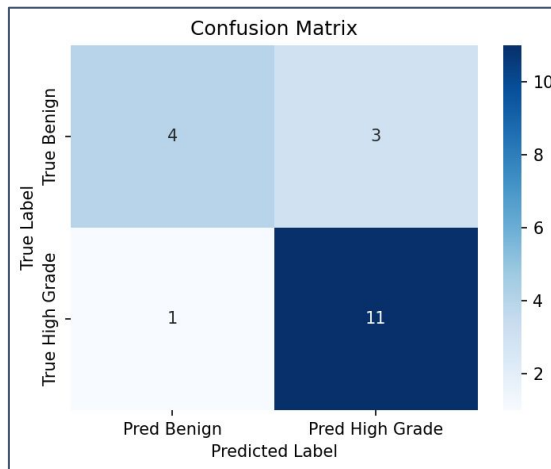
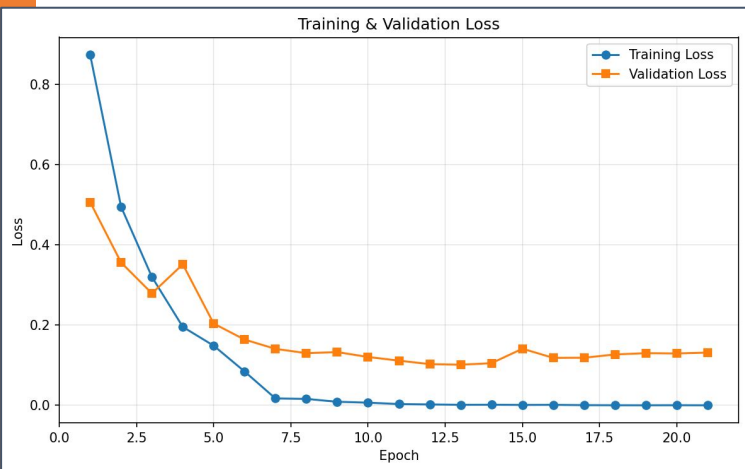
# Final Tuned Model Results:

## Round 1, Split 2

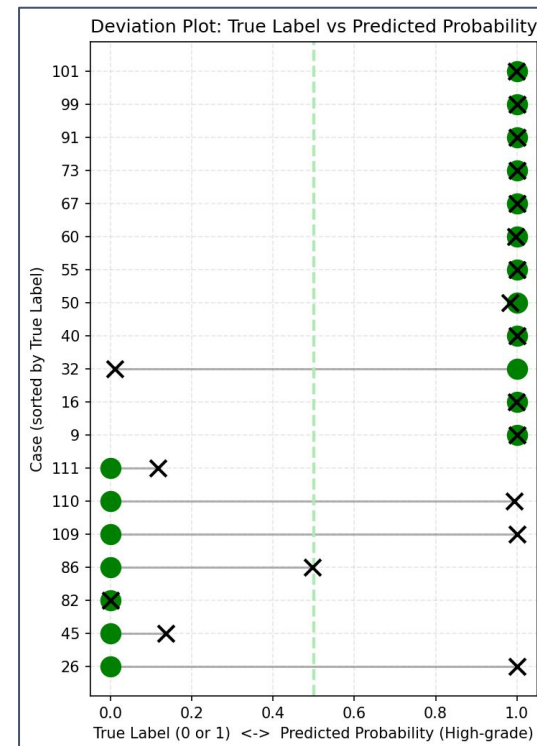
Isabella

Github:

STAT390\_Team4/Presentation\_4\_code\_Bella/split2\_vis.ipynb



- Training drops to 0 + validation levels off, then slightly goes up = learns quick, slight overfitting
- 3 false positives, 1 false negative
- Most predictions near 0 or 1, model is very confident, even in misclassifications



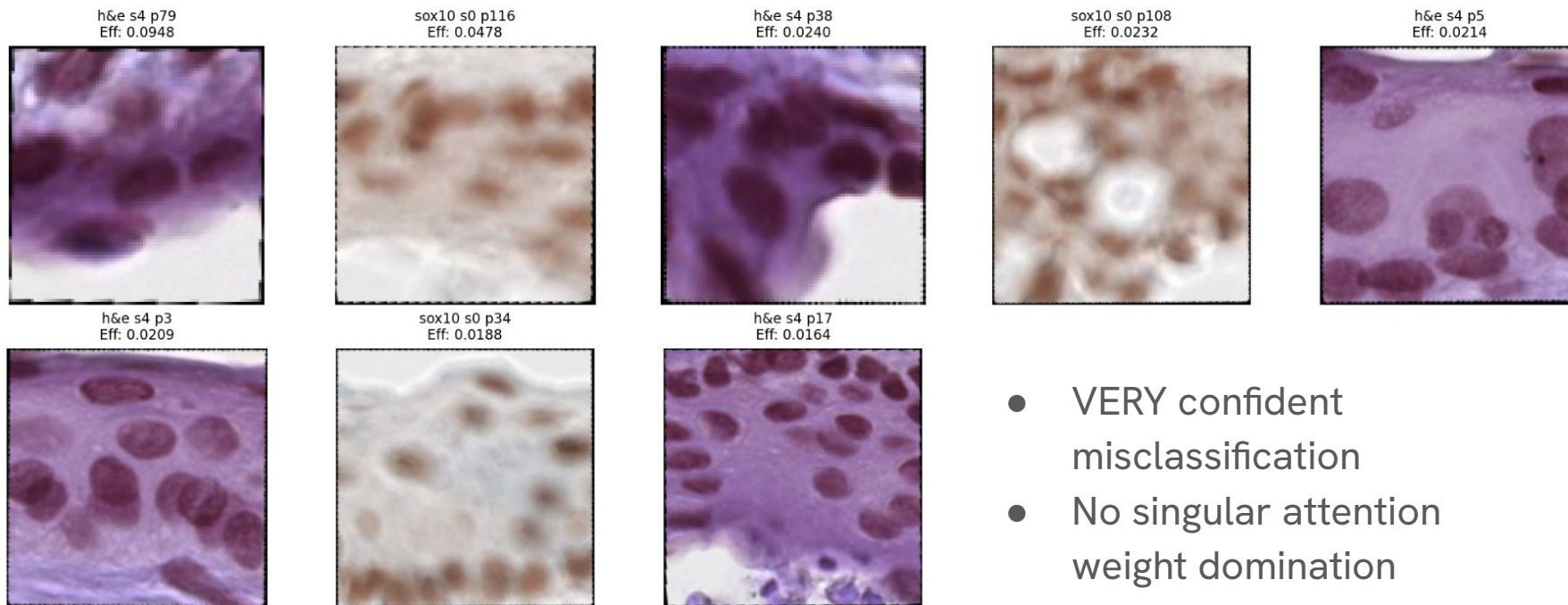
# Final Tuned Model Results: Round 1, Split 2

## Misclassified Benign- Case 26

Isabella

GitHub:  
run\_20260314\_092147/attention\_an  
alysis/case\_effective\_patches/case\_  
26\_top\_effective\_patches.png

**Case 26 - Top Effective Patches**



- VERY confident misclassification
- No singular attention weight domination

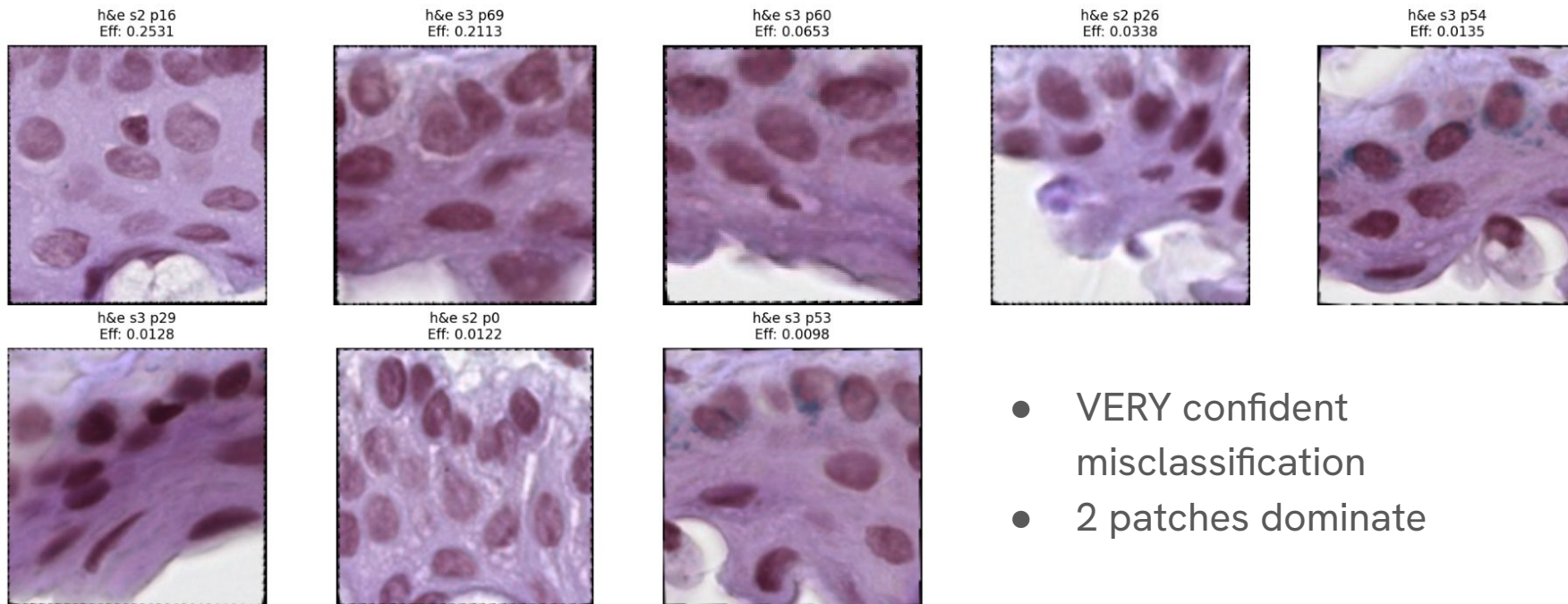
# Final Tuned Model Results: Round 1, Split 2

## Misclassified Benign- Case 109

Isabella

GitHub:  
run\_20260314\_092147/attention\_an  
alysis/case\_effective\_patches/case\_  
109\_top\_effective\_patches.png

**Case 109 - Top Effective Patches**



- VERY confident misclassification
- 2 patches dominate

# Final Tuned Model Results: Round 1, Split 2

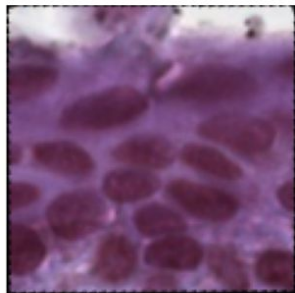
## Misclassified Benign- Case 110

Isabella

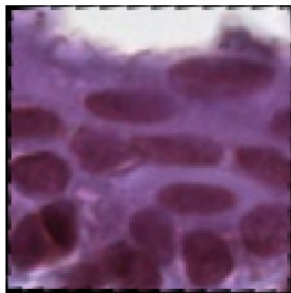
GitHub:  
run\_20260314\_092147/attention\_an  
alysis/case\_effective\_patches/case\_  
110\_top\_effective\_patches.png

**Case 110 - Top Effective Patches**

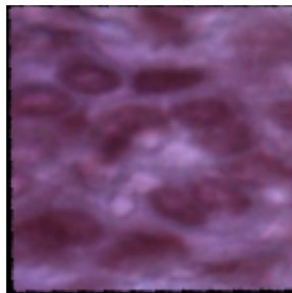
h&e s3 p60  
Eff: 0.1611



h&e s3 p61  
Eff: 0.0642



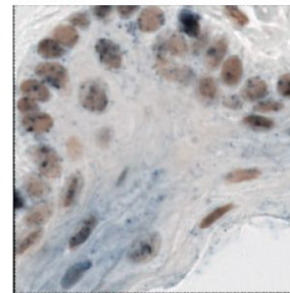
h&e s3 p85  
Eff: 0.0294



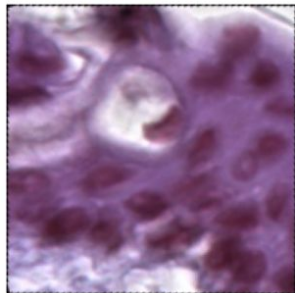
h&e s3 p14  
Eff: 0.0274



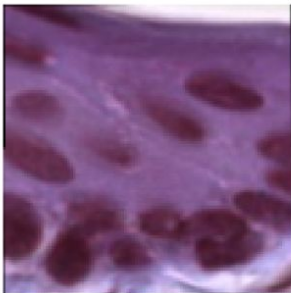
sox10 s0 p24  
Eff: 0.0222



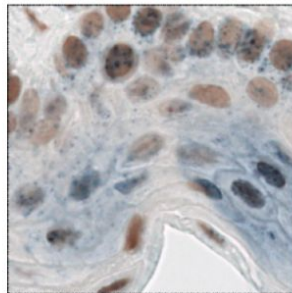
h&e s3 p100  
Eff: 0.0180



h&e s3 p66  
Eff: 0.0152



sox10 s0 p27  
Eff: 0.0149



- VERY confident misclassification
- 1 patch dominates

# Final Tuned Model Results: Round 1, Split 2

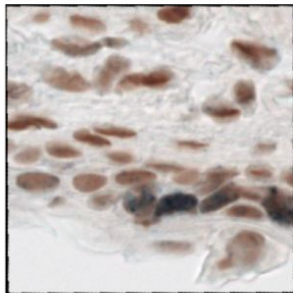
## Misclassified High Grade- Case 32

Isabella

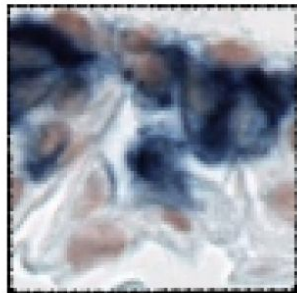
GitHub:  
run\_20260314\_092147/attention\_an  
alysis/case\_effective\_patches/case\_  
32\_top\_effective\_patches.png

**Case 32 - Top Effective Patches**

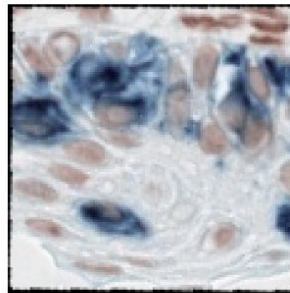
sox10 s0 p12  
Eff: 0.0249



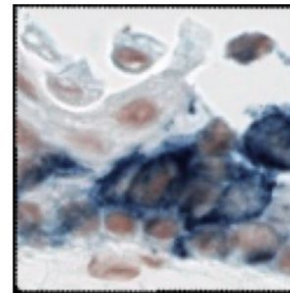
melan s0 p22  
Eff: 0.0235



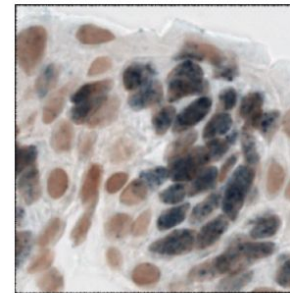
melan s0 p12  
Eff: 0.0222



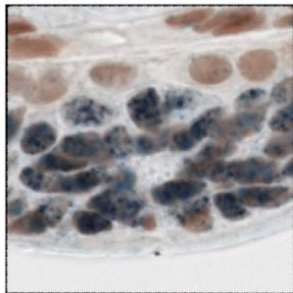
melan s1 p17  
Eff: 0.0214



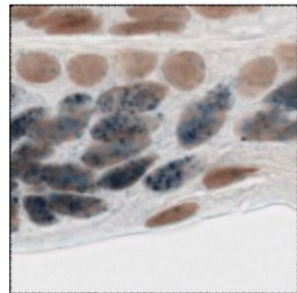
sox10 s0 p53  
Eff: 0.0157



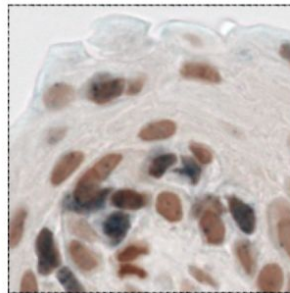
sox10 s0 p34  
Eff: 0.0145



sox10 s0 p33  
Eff: 0.0126



sox10 s0 p67  
Eff: 0.0119



- VERY confident misclassification
- No singular attention weight domination

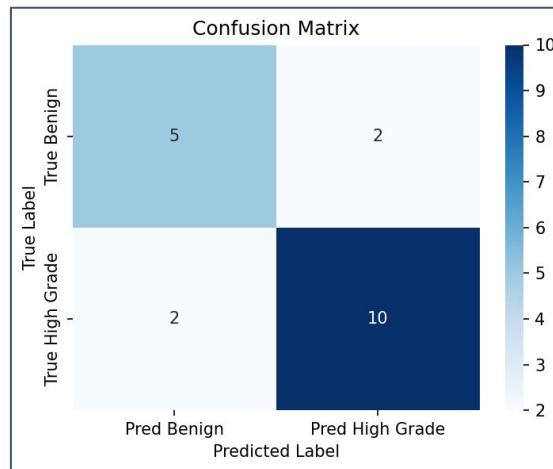
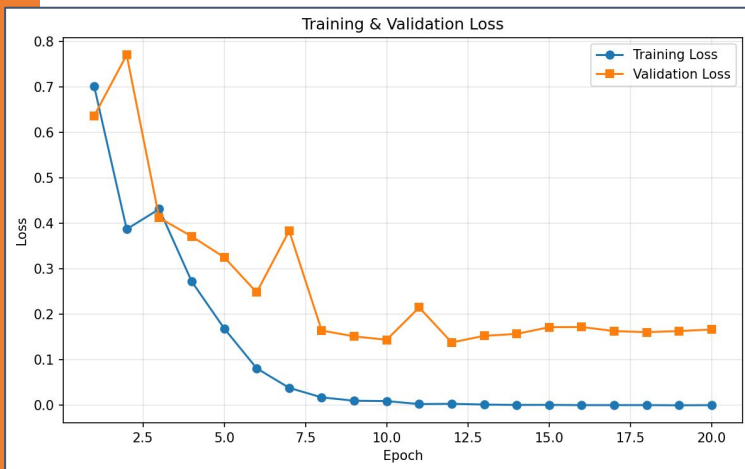
# Final Tuned Model Results:

## Round 1, Split 3

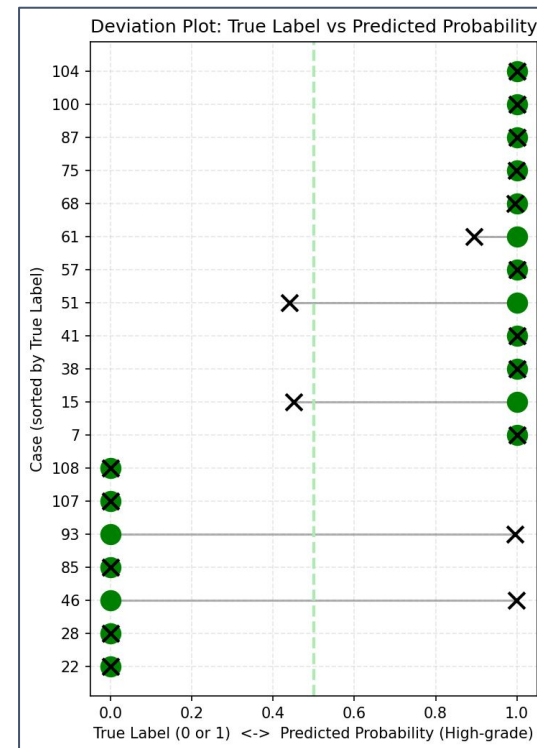
Isabella

Github:

STAT390\_Team4/Presentation\_4\_code\_Bella/split3\_vis.ipynb



- Training drops to 0 + validation goes down but then plateaus around 0.18 = slight overfitting
- 2 false positives, 2 false negatives
- Very confident in correct predictions; not confident in high-grade misclassifications, VERY confident in benign misclassifications



# Final Tuned Model Results: Round 1, Split 3

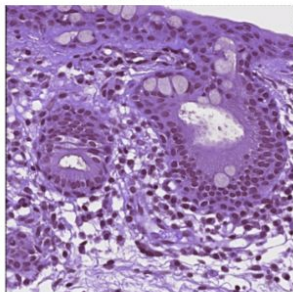
## Misclassified Benign- Case 46

Isabella

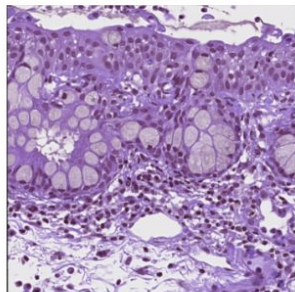
GitHub:  
run\_20260314\_093153/attention\_an  
alysis/case\_effective\_patches/case\_  
46\_top\_effective\_patches.png

**Case 46 - Top Effective Patches**

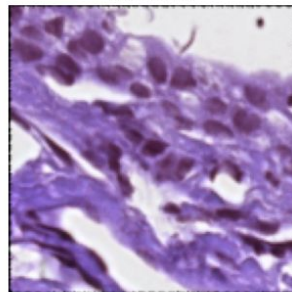
h&e s0 p4  
Eff: 0.0554



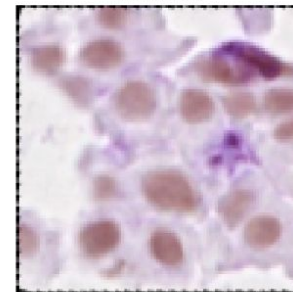
h&e s0 p32  
Eff: 0.0428



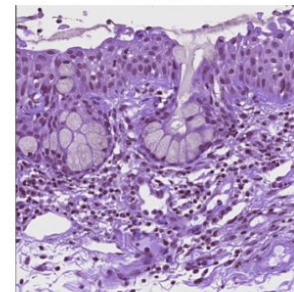
h&e s0 p14  
Eff: 0.0424



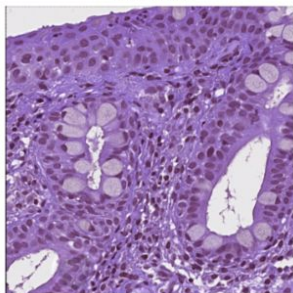
melan s0 p7  
Eff: 0.0417



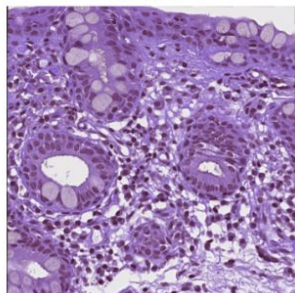
h&e s0 p31  
Eff: 0.0360



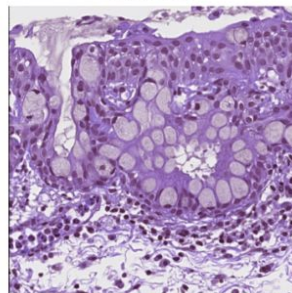
h&e s0 p9  
Eff: 0.0248



h&e s0 p5  
Eff: 0.0223



h&e s0 p33  
Eff: 0.0211



- VERY confident misclassification
- No singular attention weight domination

# Final Tuned Model Results: Round 1, Split 3

## Misclassified Benign- Case 93

Isabella

GitHub:  
run\_20260314\_093153/attention\_an  
alysis/case\_effective\_patches/case\_  
93\_top\_effective\_patches.png

**Case 93 - Top Effective Patches**

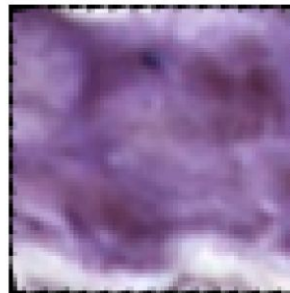
h&e s1 p56  
Eff: 0.1965



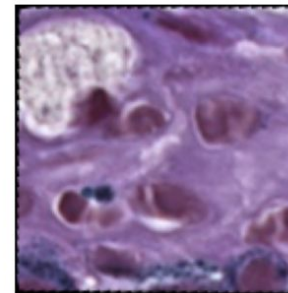
h&e s1 p94  
Eff: 0.0515



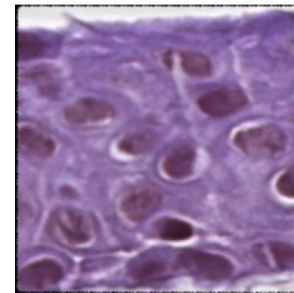
h&e s1 p47  
Eff: 0.0443



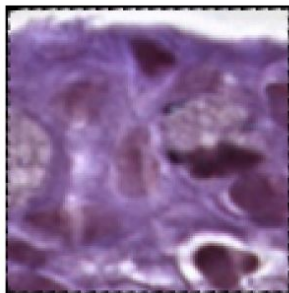
h&e s1 p21  
Eff: 0.0377



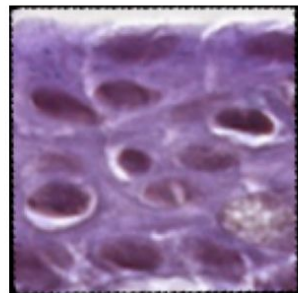
h&e s1 p28  
Eff: 0.0349



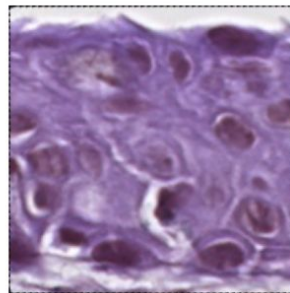
h&e s1 p37  
Eff: 0.0249



h&e s1 p12  
Eff: 0.0173



h&e s1 p29  
Eff: 0.0164



- VERY confident misclassification
- 1 patch dominates

# Final Tuned Model Results: Round 1, Split 3

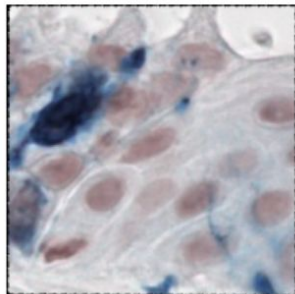
## Misclassified High Grade- Case 15

Isabella

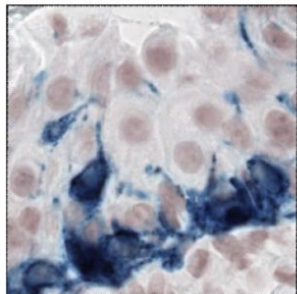
GitHub:  
run\_20260314\_093153/attention\_an  
alysis/case\_effective\_patches/case\_  
15\_top\_effective\_patches.png

**Case 15 - Top Effective Patches**

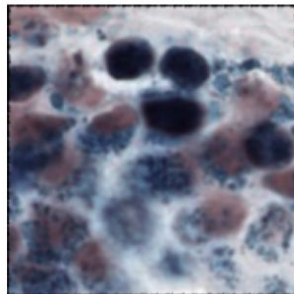
melan s3 p62  
Eff: 0.0181



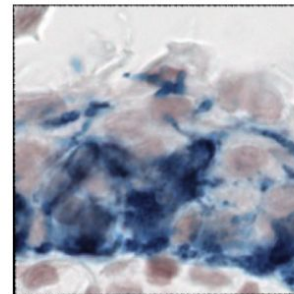
melan s3 p181  
Eff: 0.0156



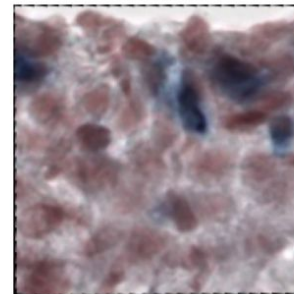
sox10 s2 p45  
Eff: 0.0132



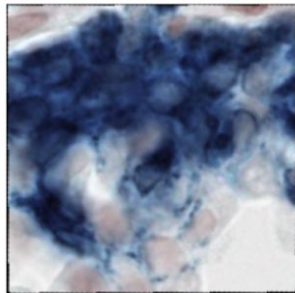
melan s3 p184  
Eff: 0.0110



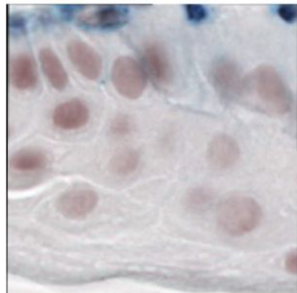
melan s2 p61  
Eff: 0.0103



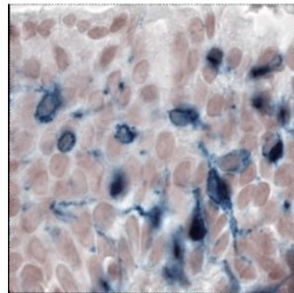
melan s3 p0  
Eff: 0.0092



melan s1 p120  
Eff: 0.0088



melan s1 p0  
Eff: 0.0087



- Not confident misclassification
- No singular attention weight domination

# Final Tuned Model Results: Round 1, Split 3

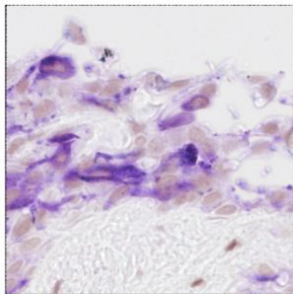
## Misclassified High Grade- Case 51

Isabella

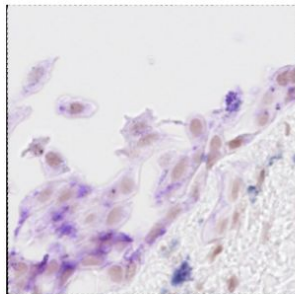
GitHub:  
run\_20260314\_093153/attention\_an  
alysis/case\_effective\_patches/case\_  
51\_top\_effective\_patches.png

**Case 51 - Top Effective Patches**

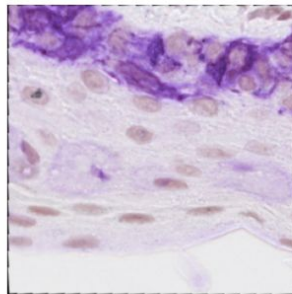
melan s1 p61  
Eff: 0.0341



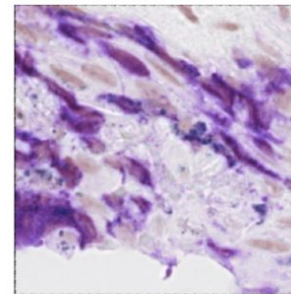
melan s1 p58  
Eff: 0.0286



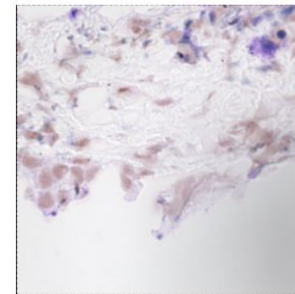
melan s0 p73  
Eff: 0.0261



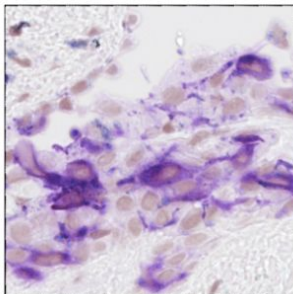
melan s0 p150  
Eff: 0.0230



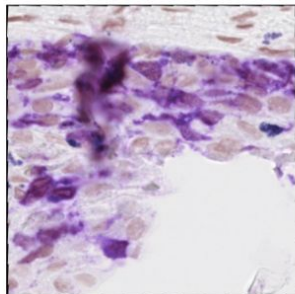
melan s1 p4  
Eff: 0.0187



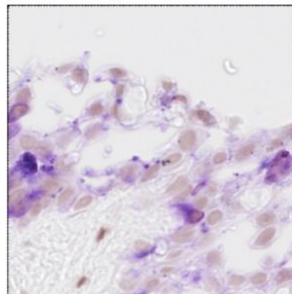
melan s1 p62  
Eff: 0.0181



melan s0 p122  
Eff: 0.0153



melan s1 p60  
Eff: 0.0147



- Not confident misclassification
- No singular attention weight domination

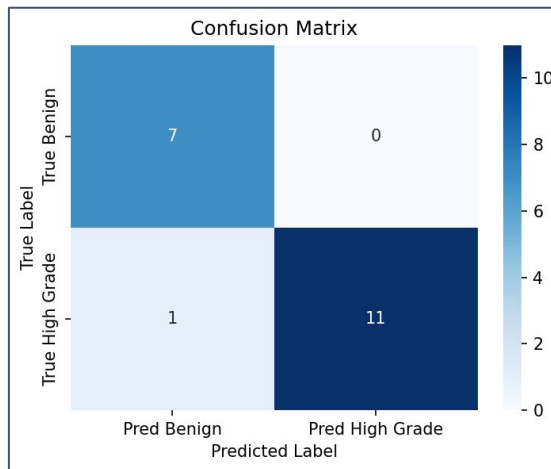
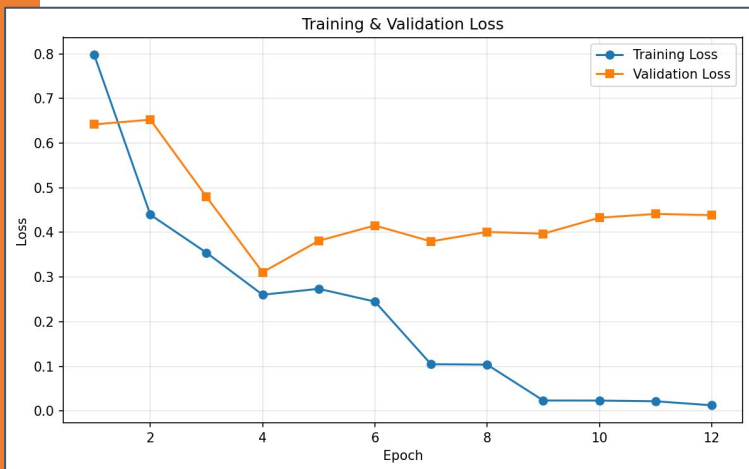
# Final Tuned Model Results:

## Round 1, Split 4

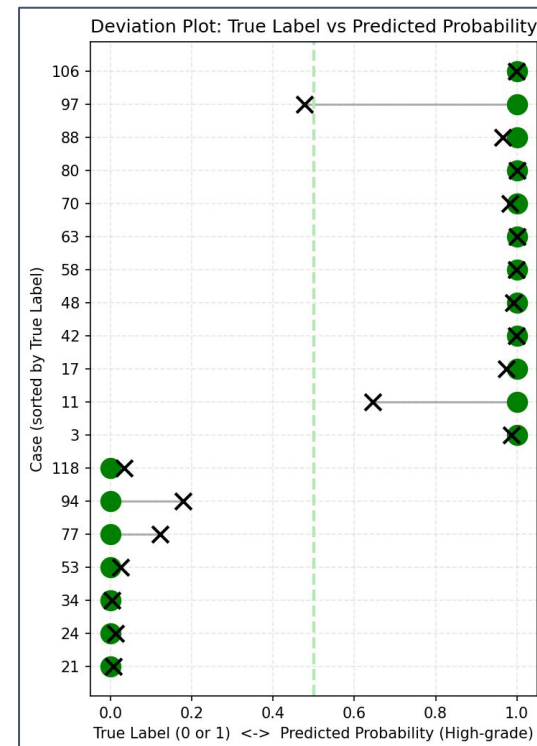
Isabella

Github:

STAT390\_Team4/Presentation\_4\_code\_Bella/split4\_vis.ipynb



- Training goes to 0 + validation goes down (lowest at 4) but starts to increase after this point = overfitting
- 0 false positives, 1 false negative
- Only 1 misclassification but not confident, most correct classifications are made with confidence



# Final Tuned Model Results: Round 1, Split 4

## Misclassified High Grade- Case 97

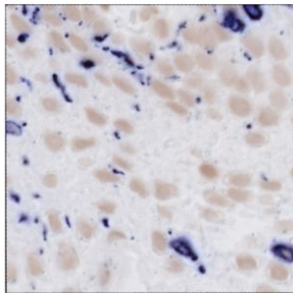
Isabella

GitHub:

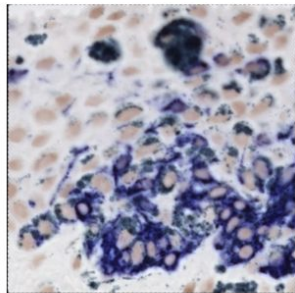
Presentation\_4\_code\_Ben/run\_2026  
0314\_094359/attention\_analysis/ca  
se\_effective\_patches/case\_97\_top\_e  
ffective\_patches.png

### Case 97 - Top Effective Patches

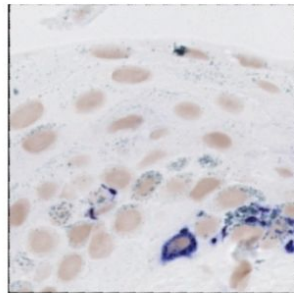
melan s0 p63  
Eff: 0.0212



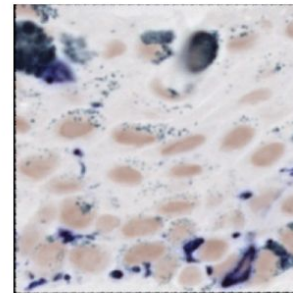
melan s0 p52  
Eff: 0.0183



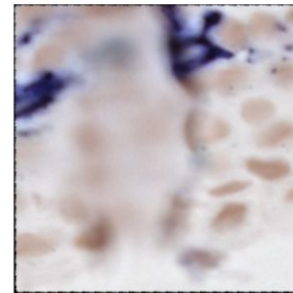
melan s1 p90  
Eff: 0.0175



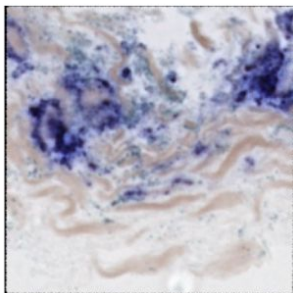
melan s1 p46  
Eff: 0.0134



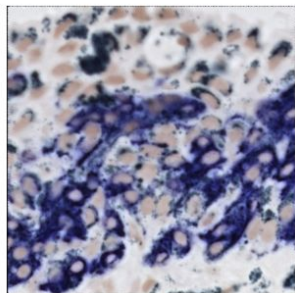
melan s0 p85  
Eff: 0.0110



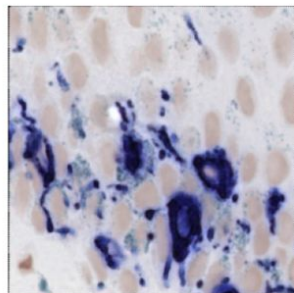
melan s1 p9  
Eff: 0.0104



melan s0 p51  
Eff: 0.0103



melan s1 p97  
Eff: 0.0098



- Not confident misclassification
- No singular attention weight domination

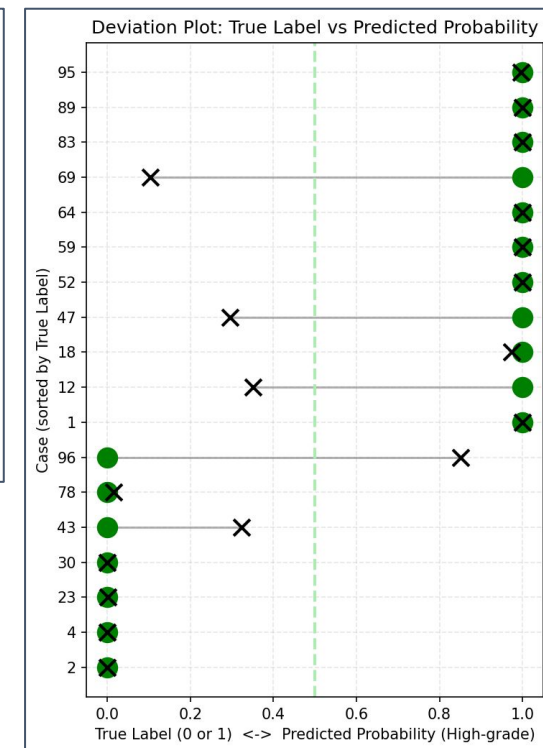
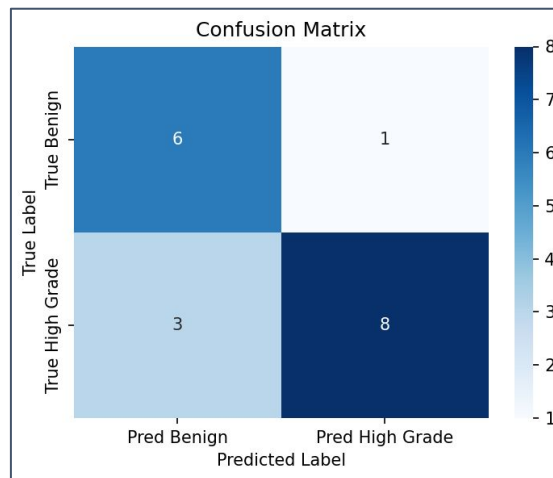
# Final Tuned Model Results:

## Round 1, Split 5

Isabella

Github:

STAT390\_Team4/Presentation\_4\_code\_Bella/split5\_vis.ipynb



- High instability in validation loss early on; big spike
- Training loss goes to zero + validation increases and stays high = severe overfitting after around epoch 6
- 1 false positive, 3 false negatives
- Moderately confident misclassification for cases 96 and 69, not confident in 47 and 12. Confident in most correct classifications

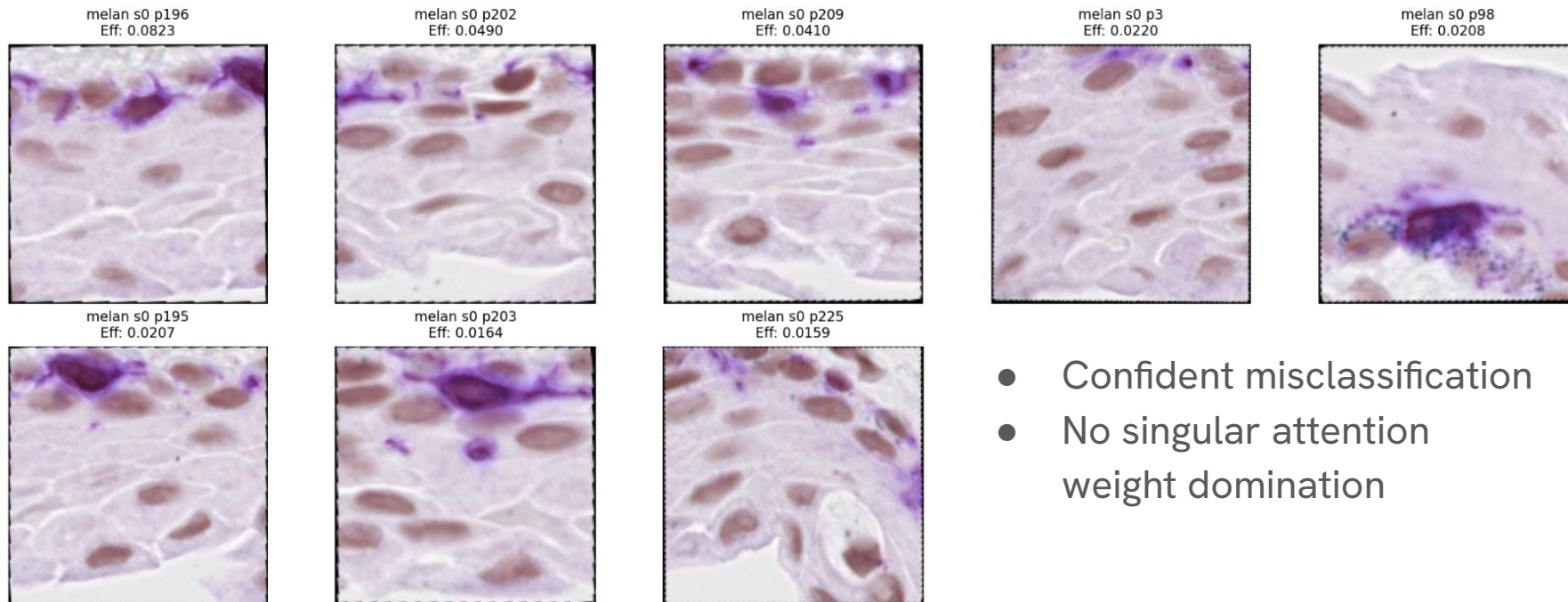
# Final Tuned Model Results: Round 1, Split 5

## Misclassified Benign- Case 96

Isabella

GitHub:  
run\_20260314\_095526/attention\_an  
alysis/case\_effective\_patches/case\_  
96\_top\_effective\_patches.png

**Case 96 - Top Effective Patches**



- Confident misclassification
- No singular attention weight domination

# Final Tuned Model Results: Round 1, Split 5

## Misclassified High Grade- Case 12

Isabella

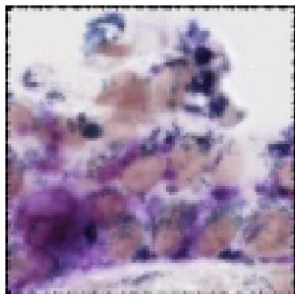
GitHub:  
run\_20260314\_095526/attention\_an  
alysis/case\_effective\_patches/case\_  
12\_top\_effective\_patches.png

**Case 12 - Top Effective Patches**

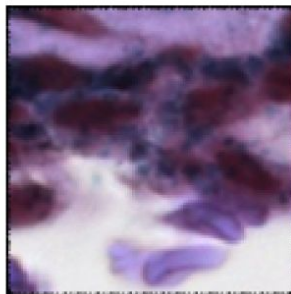
h&e s0 p28  
Eff: 0.0863



melan s1 p40  
Eff: 0.0514



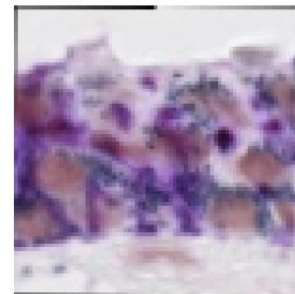
h&e s0 p24  
Eff: 0.0268



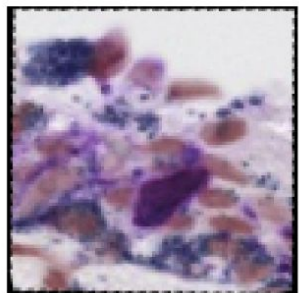
melan s0 p19  
Eff: 0.0222



melan s1 p36  
Eff: 0.0213



melan s0 p1  
Eff: 0.0199



h&e s0 p1  
Eff: 0.0179



melan s0 p18  
Eff: 0.0176



- Not confident misclassification
- No singular attention weight domination

# Final Tuned Model Results: Round 1, Split 5

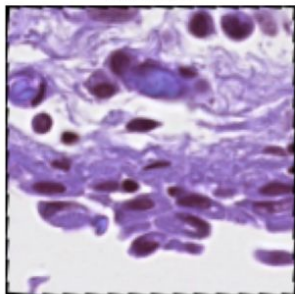
## Misclassified High Grade- Case 47

Isabella

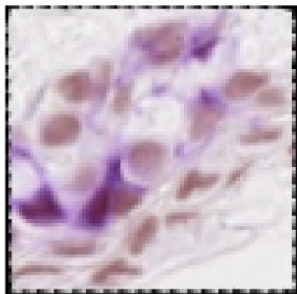
GitHub:  
run\_20260314\_095526/attention\_an  
alysis/case\_effective\_patches/case\_  
47\_top\_effective\_patches.png

**Case 47 - Top Effective Patches**

h&e s0 p303  
Eff: 0.0289



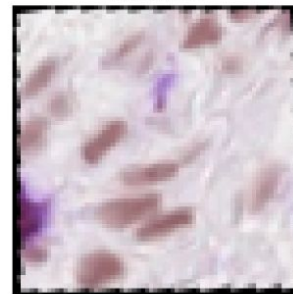
melan s0 p45  
Eff: 0.0252



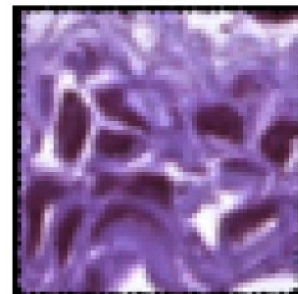
melan s0 p44  
Eff: 0.0244



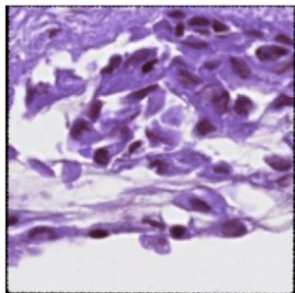
melan s0 p50  
Eff: 0.0205



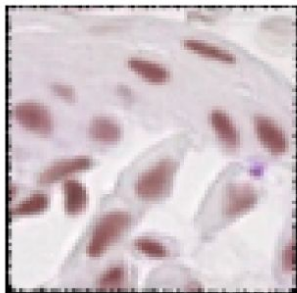
h&e s0 p92  
Eff: 0.0135



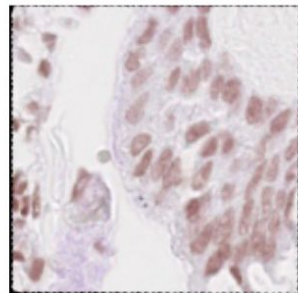
h&e s0 p312  
Eff: 0.0096



melan s0 p14  
Eff: 0.0095



melan s0 p202  
Eff: 0.0078



- Not confident misclassification
- No singular attention weight domination

# Final Tuned Model Results: Round 1, Split 5

## Misclassified High Grade- Case 69

Isabella

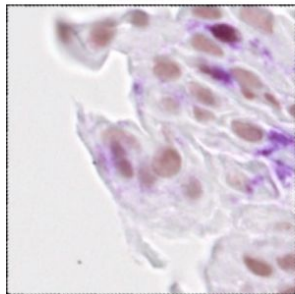
GitHub:  
run\_20260314\_095526/attention\_an  
alysis/case\_effective\_patches/case\_  
69\_top\_effective\_patches.png

**Case 69 - Top Effective Patches**

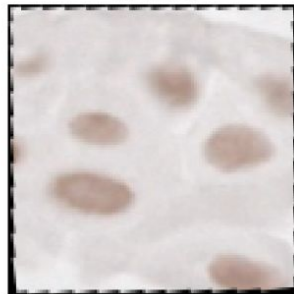
sox10 s3 p21  
Eff: 0.0408



melan s2 p4  
Eff: 0.0364



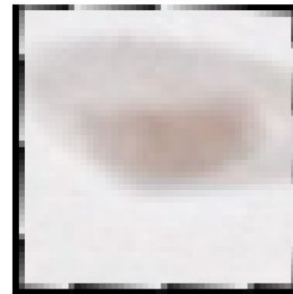
sox10 s1 p6  
Eff: 0.0265



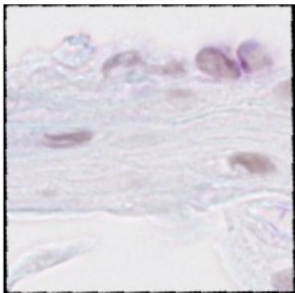
sox10 s3 p16  
Eff: 0.0206



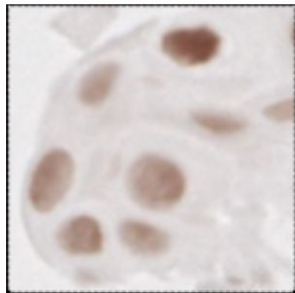
sox10 s1 p5  
Eff: 0.0203



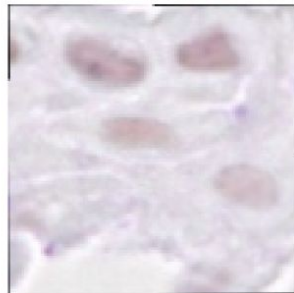
melan s0 p1  
Eff: 0.0200



sox10 s1 p3  
Eff: 0.0165



melan s1 p7  
Eff: 0.0163



- Confident misclassification
- No singular attention weight domination

# Personal Reflection: Isabella

Isabella

## What Did You Learn? [Technical Learnings]

### Using QUEST Effectively

Gained hands-on experience working with (and troubleshooting) Northwestern's high-performance computing cluster for model training and evaluation

### Building & Interpreting Performance Evaluation Graphs

Learned how to create and interpret visualizations (i.e. val/train loss vs. epochs, confusion matrices, deviation plots, high-attention patches, etc.) to assess model performance across runs and split.

### Understanding Attention Mechanisms

Built intuition for attention weights and how models prioritize different inputs, especially in understanding which features or regions drive predictions.

### Importance of Hyperparameter Tuning

Recognized how critical hyperparameter tuning is for model performance, and experienced how computationally intensive and time-consuming it can be

## What Did You Learn? [Non-Technical Learnings]

### Effective Online Communication

Importance of responsiveness, conciseness, and clarity when communicating online in team environments, ensuring smoother collaboration.

### Explaining Work & Answering Questions

Gained confidence in answering questions, explaining work, and presenting technical concepts clearly to others.

### Teamwork & Coordination

Learned how to divide tasks, align on goals, and maintain progress within a group setting.

# Personal Reflection: Isabella

Isabella

## What Would You Do Differently In The Future?

### **Start Work Earlier!**

Begin projects sooner to account for unexpected issues such as long Quest “Pending” queues and long model training runtimes.

### **Communicate More Concisely**

Focus on delivering clearer, more direct arguments and explanations, particularly in presentations and written reports.

## What Will You Apply in Real-World Data Science Projects?

### **Version Control & Collaboration (GitHub)**

Use GitHub for organizing code, tracking changes, and collaborating efficiently in team-based environments.

### **Structured Workflows & Documentation**

Implement organized, reproducible workflows across files and work to ensure scalability and clarity in projects.

### **Professional Communication**

Apply skills writing clear emails to supervisors (i.e. Krish/Aryaman), setting expectations, and communicating timelines effectively with team members.

# **Ben Personal Reflection**

# Technical Learnings



## Quest + VSCode

Mastered HPC commands, sbatch files, and remote code execution on Northwestern's Quest cluster.



## MIL Deep Learning

Built a hierarchy-aware model across Patches → Stains → Cases for whole-slide cancer image analysis.



## OPTUNA Optimization

Designed efficient hyperparameter studies using Sobol + OPTUNA with pruning to cut wasted computation time.



## Early Stopping

Gained deep intuition for how patience, delta, and scheduler variables control training duration.



## DenseNet / KimiaNet

First time using a pretrained model — replaced building from scratch with transfer learning.

# Beyond the Code

## Non-Technical Learnings

### Logistics & Running Code

Learned the practical realities of scheduling, queuing, and debugging jobs on an HPC cluster.

### Pacing & Effort Distribution

Recognized the cost of burst-working. Would replace long weekend sessions with 1-hour daily blocks — spreading effort consistently across the quarter.

## What I'd Do Differently

### Daily Code Sessions

Allocate ~1 hour each day to run code and review outputs — rather than spending multiple hours in a single weekend session before the next presentation.

### Proactive GitHub Management

Communicate with teammates before committing, actively check branch status, and watch for merge conflicts before every push. Two incidents taught me this the hard way.

# Skills I'll Carry Forward

*Learnings that will directly shape my future work in industry or research:*

01

## AI-Assisted Problem Solving

Using LLMs to brainstorm code improvements. Example: Claude suggested caching patch embeddings — we implemented it, dramatically cutting runtime.

02

## Multiple Instance Learning

MIL is broadly applicable wherever hierarchical data exists — more cancer research, clinical datasets, or any domain with grouped, unlabeled instances.

03

## GitHub & Team Collaboration

Lived through two major repo incidents. Now acutely aware of merge conflict risks, branch hygiene, and the discipline required for multi-contributor codebases.

### Technical Learnings

#### Understanding MIL models and deep learning pipelines

Although I did not do any work on implementation, I learned how data flows from images to many cycles to get an end result. The math that goes behind feature extraction, attention pooling/ weighting, and the final prediction layers. This understanding helps immensely with reasoning with model behavior.

#### How dropout works in different areas of the model

Utilizing certain regularization techniques like dropout in the incorrect areas of the model can have a massive implication on how the neural network operates. Including it in different areas (in softmax, attention pooling, classifier stage) have different effects on interpretability

#### Impact of hyperparameters

Image resolution, patch size, learning rate, dropout, etc. all play a massive role in building a highly accurate prediction model as they can help fit to the bias variance tradeoff quite well. Beyond that computationally intensive models hurt a project in the long run as it gives less time to help find a solution.

#### Cost of a model

To add on to the last point made, making a model that is accurate and has limited redundancies is vital to its growth in the long run. If the model is very expensive and takes a long time to train/test, or there are multiple useless processes in the model, training times can shoot through the roof especially when working with large real world datasets.

### Non-Technical Learnings

#### Structured processes and organization

Everyone on the team knowing their objectives and roles throughout the week was extremely helpful. It made it much easier to work on things when there is an end goal in mind and there are specific times for when things should be accomplished.

#### Effective communication

The importance of this can not be understated. It was very easy to get a hold of anyone in the group which made answering questions much easier. This helped root out any redundancy in the presentation as well as build off each others ideas to create a more innovative approach/presentation.

#### Operating on incomplete/creative instructions

In a perfect world, knowing exactly what we need to do is the goal, but often times it was the case where something needed to get done without a perfect process for how it should be completed. I realized instead of waiting for confirmation, it is best to come up with a solution to the problem that can be easily modified in case there is an important piece missed.

# Personal Reflection: Akshay Anand continued...

---

## What would I do differently?

- 1.) Improve my system for tracking and implementing changes
  - a.) Make changes one at a time
  - b.) Prioritize reproducibility of said changes to help with debugging and refinement of the code
  - c.) Report on the effects of each change instead of proposing 3 changes based on mathematical theory
- 2.) Communicate more with the coders early on to help design/create these mini experiments so that our findings are not just based on the accuracy number produced, but specific causation as to what changed.

## Real World Application

- 1.) Given that the project was done in medical imaging, the stakes are high and every bit of improvement in the model is incredibly important. Understanding the level of detail that needs to be placed on improving the model is something I will take to future projects
- 2.) Mathematical processes used throughout the model indicate that neural networks have the ability to fit to these complex patterns, but need significant constraints to make them functional.
- 3.) Often times in large models, it can be easy to get lost and confuse what is happening. Understanding the mathematical theory behind the methods used is something I will apply to future large ML projects.

# Small Data Memorization in MIL Models

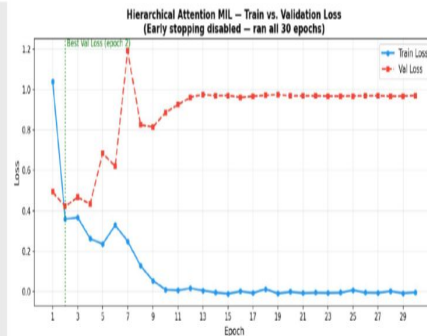
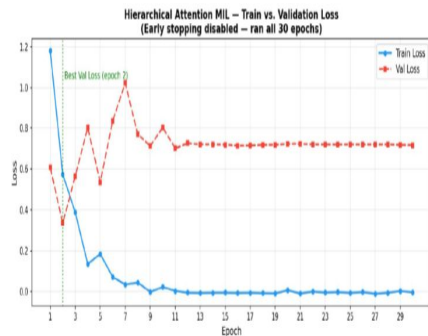
Akshay Anand

Objective:

1: Why overparameterized models can memorize a small training set and drive training loss to 0 while validation loss remains very high?

AND

2: Why does the problem mentioned above persist as a small-data issue even while the patch sizes approach massive sizes such as infinity?



Why does the situation above exist when there exist fewer than 100 cases, but each case yields 500-1500 patches?

# Objective 1

How does an overparameterized model move training loss to 0 while validation loss remains high?

$AB=y$  where  $A \in \mathbb{R}^{n \times p}$  data matrix  
 $n$ : samples,  $p$ : parameters  
 $y \in \mathbb{R}^n$ , labels  $\theta \in \mathbb{R}^p$ , parameters

If the  $p$  value exceeded the  $n$  value  
 $p > n$ ,  
then we would have more parameters  
than equations.

If our  $p > n$  and we let  
 $\text{rank}(A) = r \leq n$ , the  $\dim(\text{Null}(A)) = p-r$

Since  $p > n \geq r$ , we yield  $p-r > 0$

$\therefore$  The null space is non-trivial  
and there exists a non-zero vector  
 $v \neq 0$  such that  $Av = 0$

If we can change the parameters  
without changing the predictions, we  
can conclude the system has  
infinite solutions

Example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix} \quad y = \begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

$$AB=y$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix} = \begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

$$1\theta_1 + 2\theta_2 + 3\theta_3 = 7$$

$$4\theta_1 + 5\theta_2 + 6\theta_3 = 8$$

Apply Rank-Nullity theorem

$$\text{Rank}(A) + \text{Nullity}(A) = p \text{ (parameters)}$$

$$2 + \text{Nullity}(A) = 3$$

$\therefore$

$$\text{Nullity}(A) = 1$$

To solve, we must set:

$$\theta_3 = 6 \leftarrow \text{can use any } \theta_n$$

which yields a solution of

$$\theta = \begin{bmatrix} \theta_1(t) \\ \theta_2(t) \\ 6 \end{bmatrix}$$

Evident of infinite solutions

- This is a mathematical proof that indicates when the # of parameters exceeds the number of equations/sample size, infinite solutions are found
- Both a theoretical/conceptual explanation as well as an example are provided

# How does the proof of infinite solutions when $p > n$ prove our main objective?

- If  $p > n$ , then our initial system of  $A(\theta) = y$  has infinite solutions which means that there is many  $\theta$  to satisfy a the equation perfectly
- If there are infinite solutions, then there is no discrepancy between prediction and the label
- Given that there are solutions that will yield 0 (absolute minimum), the models will learn on those and just memorize the training data

Training problem:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i)$$

We choose  $\theta$  to help predictions match labels on the training data

To minimize loss, we must

$$f_{\theta}(x_i) \approx y_i \quad \forall i$$

or

$$f_{\theta}(x_i) = y_i \quad \forall i$$

$\infty$   
 $\theta$  such that  $f_{\theta}(x_i) = y_i \quad \forall i$

$$\ell(f_{\theta}(x_i), y_i) = 0$$

which  $\therefore L_{\text{train}} = 0$

# Objective 2

Why does the problem remain effectively small data when case numbers are small, but number of patches approaches infinity?

Hierarchy of the data

Case  $\rightarrow$  Slice

$\hookrightarrow$  Slice

$\hookrightarrow$  Patch

$B_i$  = whole case

$S_{ik}$  = slice w/in case  $i$

$L_{ikl}$  = slice w/in ~~case~~ <sup>slice</sup>  $k$

$$B_i = \{S_{i1}, S_{i2}, \dots, S_{iK_i}\} \quad \text{Xikl is patch within slice } l$$

$$S_{ik} = \{L_{ik1}, L_{ik2}, \dots, L_{ikM_{ik}}\}$$

$$L_{ikl} = \{X_{ikl1}, X_{ikl2}, \dots, X_{iklN_{ikl}}\}$$

$y_i \in \{0, 1\}$   $\leftarrow$  this label applies to  
the ENTIRE case  $B_i$   
not to any sub levels

Setting the stage:

- The way in which the data is organized, labels are only placed at the case level, not at the patch level
- Given that patches are not independently labeled, they do not create any new supervised samples

## Objective 2 continued...

- As shown in the mathematical proof, it is evident that the loss function only operates for every case, not every patch
- Therefore, a sheer increase in the number of patches will have no effect on the number of cases involved
- An increase in patch size does not affect the number of samples (n from linear algebra proof) and therefore will yield infinite solutions

How the model maps:

$$\hat{y}_i = F_{\theta}(B_i)$$

Where  $F_{\theta}$ : neural network with parameters  $\theta$

Takes all the patches to one case and outputs a prediction

Then we look at the loss function:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(F_{\theta}(B_i), y_i)$$

Note how we are calculating loss once per case and patches are NOT independently supervised

# Concluding Objective 2...

Infinity problem

Consider case in which patch value approaches infinity:

-  $m_i \rightarrow \infty$        $N = \# \text{ of cases}$

$N$  is still finite as the patch size increasing or decreasing does NOT affect case number.

- Even if patch size were to reach a massive number, the number of labeled samples would not change
- This would lead to the effective size of the dataset to remain small  $\rightarrow$  small-data issue
- As long as the within case data grows without supervision/labeling, the dataset will always remain the same size

# Assumptions for my arguments

1: Overparameterization - where the number of parameters exceeds the number of samples

$$p > n$$

2: Case-level supervision/labeling - Labeling only occurs at the case level and not within the patches themselves

3: Independence of samples - Each case is considered one restraint

4: Model approximation - The neural network can approximate arbitrary functions when there is a proper number of parameters and samples

Following these assumptions, it can be proven that the effective sample size of the model is based in the number of labeled cases, and independent of the number of patches within each case. This indicates that even as the number of patches grows to extremely high numbers, the problem remains in the small data region as the number of constraints remain the same. As a result of this issue, the model may memorize training cases rather than learning the vital generalizable patterns that exist across the data.

# Coder Objective 2: Effect of patch order on accuracy

Zoe

## main.py changes (link to github added)

### Multi-run experiment pipeline

- Added --multi\_run, --splits\_dir, --seeds, --aggregate\_output\_dir
- Looping logic in main() to run:
  - all splits via discover\_split\_files()
  - all seeds via nested loop

### New helper functions

- discover\_split\_files() → finds all .npz split files
- get\_split\_id() → creates consistent split labels
- run\_one() → runs training + evaluation for one (split, seed)
- results\_to\_prediction\_df() → formats predictions into per-case rows
- aggregate\_and\_save() → aggregates and saves final CSV outputs

### Evaluation Improvements

- Reload best checkpoint before testing: trainer.load\_checkpoint(best\_ckpt)
- Ensures test results reflect best validation performance, not last epoch

### Created inside aggregate\_and\_save():

- best\_val\_loss\_boxplot\_ready.csv
- predicted\_probabilities\_boxplot\_ready.csv
- classification\_consistency.csv
- accuracy\_variability.csv

# Coder Objective 2: Effect of patch order on accuracy

Zoe

## [trainer.py](#) changes (link to github added)

Best model tracking

- Track best validation loss: `self.best_val_loss`
- Store best checkpoint path: `self.best_checkpoint_path`

Training loop updates:

- Inside `train()`:
  - Save checkpoint when validation improves
  - Return summary dictionary:

```
{  
    "best_val_loss": ...,  
    "best_checkpoint_path": ...  
}
```

# Effect of patch order on accuracy:

## Accuracy Variability

Accuracy is generally high across all splits

However, there is noticeable variability across random seeds

Some splits (e.g., split 2) are more sensitive to patch order

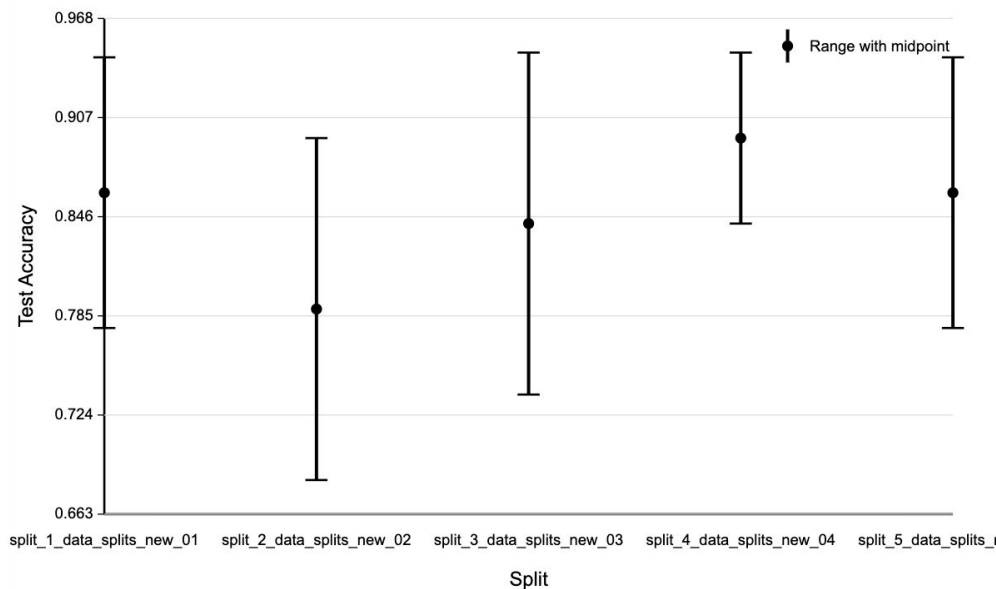
Others (e.g., split 4) are more stable

Midpoints are consistent → typical performance is reliable across splits

Zoe

| split_id                   | min_test_accuracy  | max_test_accuracy  |
|----------------------------|--------------------|--------------------|
| split_1_data_splits_new_01 | 0.7777777777777778 | 0.9444444444444444 |
| split_2_data_splits_new_02 | 0.6842105263157895 | 0.8947368421052632 |
| split_3_data_splits_new_03 | 0.7368421052631579 | 0.9473684210526315 |
| split_4_data_splits_new_04 | 0.8421052631578947 | 0.9473684210526315 |
| split_5_data_splits_new_05 | 0.7777777777777778 | 0.9444444444444444 |

Accuracy Variability Across Splits



# Effect of patch order on accuracy:

## Classification Consistency

Zoe

| split_id                   | case_id | pct_correct | split_2_data_splits_new_02 | 9   | 100.0 | split_3_data_splits_new_03 | 7   | 100.0 | split_4_data_splits_new_04 | 3   | 80.0  | split_5_data_splits_new_05 | 1  | 90.0  |
|----------------------------|---------|-------------|----------------------------|-----|-------|----------------------------|-----|-------|----------------------------|-----|-------|----------------------------|----|-------|
| split_1_data_splits_new_01 | 5       | 100.0       | split_2_data_splits_new_02 | 16  | 100.0 | split_3_data_splits_new_03 | 15  | 20.0  | split_4_data_splits_new_04 | 11  | 90.0  | split_5_data_splits_new_05 | 2  | 100.0 |
| split_1_data_splits_new_01 | 14      | 100.0       | split_2_data_splits_new_02 | 26  | 0.0   | split_3_data_splits_new_03 | 22  | 100.0 | split_4_data_splits_new_04 | 17  | 90.0  | split_5_data_splits_new_05 | 4  | 100.0 |
| split_1_data_splits_new_01 | 19      | 90.0        | split_2_data_splits_new_02 | 32  | 40.0  | split_3_data_splits_new_03 | 28  | 100.0 | split_4_data_splits_new_04 | 21  | 100.0 | split_5_data_splits_new_05 | 12 | 90.0  |
| split_1_data_splits_new_01 | 25      | 100.0       | split_2_data_splits_new_02 | 40  | 100.0 | split_3_data_splits_new_03 | 38  | 100.0 | split_4_data_splits_new_04 | 24  | 100.0 | split_5_data_splits_new_05 | 18 | 30.0  |
| split_1_data_splits_new_01 | 27      | 80.0        | split_2_data_splits_new_02 | 45  | 20.0  | split_3_data_splits_new_03 | 41  | 100.0 | split_4_data_splits_new_04 | 34  | 100.0 | split_5_data_splits_new_05 | 23 | 100.0 |
| split_1_data_splits_new_01 | 36      | 100.0       | split_2_data_splits_new_02 | 50  | 100.0 | split_3_data_splits_new_03 | 46  | 20.0  | split_4_data_splits_new_04 | 42  | 100.0 | split_5_data_splits_new_05 | 30 | 100.0 |
| split_1_data_splits_new_01 | 44      | 100.0       | split_2_data_splits_new_02 | 55  | 100.0 | split_3_data_splits_new_03 | 51  | 60.0  | split_4_data_splits_new_04 | 48  | 100.0 | split_5_data_splits_new_05 | 43 | 100.0 |
| split_1_data_splits_new_01 | 49      | 100.0       | split_2_data_splits_new_02 | 60  | 100.0 | split_3_data_splits_new_03 | 57  | 100.0 | split_4_data_splits_new_04 | 53  | 90.0  | split_5_data_splits_new_05 | 47 | 50.0  |
| split_1_data_splits_new_01 | 54      | 100.0       | split_2_data_splits_new_02 | 67  | 100.0 | split_3_data_splits_new_03 | 61  | 50.0  | split_4_data_splits_new_04 | 58  | 100.0 | split_5_data_splits_new_05 | 52 | 100.0 |
| split_1_data_splits_new_01 | 56      | 10.0        | split_2_data_splits_new_02 | 73  | 100.0 | split_3_data_splits_new_03 | 68  | 100.0 | split_4_data_splits_new_04 | 63  | 100.0 | split_5_data_splits_new_05 | 59 | 100.0 |
| split_1_data_splits_new_01 | 66      | 100.0       | split_2_data_splits_new_02 | 82  | 100.0 | split_3_data_splits_new_03 | 75  | 60.0  | split_4_data_splits_new_04 | 70  | 100.0 | split_5_data_splits_new_05 | 64 | 100.0 |
| split_1_data_splits_new_01 | 72      | 90.0        | split_2_data_splits_new_02 | 86  | 50.0  | split_3_data_splits_new_03 | 85  | 100.0 | split_4_data_splits_new_04 | 77  | 90.0  | split_5_data_splits_new_05 | 69 | 70.0  |
| split_1_data_splits_new_01 | 79      | 80.0        | split_2_data_splits_new_02 | 91  | 100.0 | split_3_data_splits_new_03 | 87  | 100.0 | split_4_data_splits_new_04 | 80  | 100.0 | split_5_data_splits_new_05 | 78 | 90.0  |
| split_1_data_splits_new_01 | 84      | 70.0        | split_2_data_splits_new_02 | 99  | 100.0 | split_3_data_splits_new_03 | 93  | 70.0  | split_4_data_splits_new_04 | 88  | 100.0 | split_5_data_splits_new_05 | 83 | 100.0 |
| split_1_data_splits_new_01 | 90      | 100.0       | split_2_data_splits_new_02 | 101 | 100.0 | split_3_data_splits_new_03 | 100 | 100.0 | split_4_data_splits_new_04 | 94  | 20.0  | split_5_data_splits_new_05 | 89 | 100.0 |
| split_1_data_splits_new_01 | 92      | 60.0        | split_2_data_splits_new_02 | 109 | 80.0  | split_3_data_splits_new_03 | 104 | 100.0 | split_4_data_splits_new_04 | 97  | 80.0  | split_5_data_splits_new_05 | 95 | 100.0 |
| split_1_data_splits_new_01 | 98      | 100.0       | split_2_data_splits_new_02 | 110 | 100.0 | split_3_data_splits_new_03 | 107 | 100.0 | split_4_data_splits_new_04 | 106 | 100.0 | split_5_data_splits_new_05 | 96 | 50.0  |
| split_1_data_splits_new_01 | 119     | 100.0       | split_2_data_splits_new_02 | 111 | 50.0  | split_3_data_splits_new_03 | 108 | 100.0 | split_4_data_splits_new_04 | 118 | 100.0 | split_5_data_splits_new_05 |    |       |

# Effect of patch order on accuracy:

## Classification Consistency

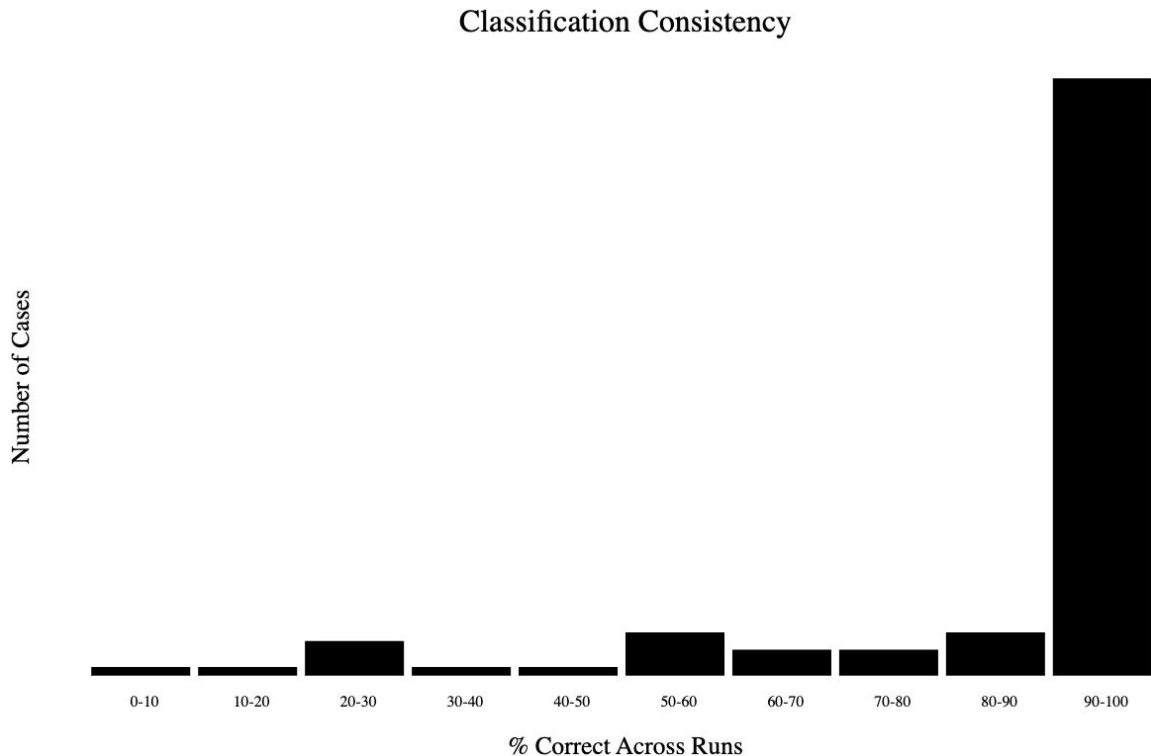
Zoe

Most cases show high consistency (90–100%)

Indicates strong stability across random seeds

A small subset of cases shows low consistency (<80% correct):

These likely represent difficult or borderline examples



# Effect of patch order on accuracy:

## Low Consistency Cases

Zoe

Total low-consistency cases: **19**

Most instability comes from:

**Split 2 and Split 3**

Split 4 is very stable (only 1 bad case)

A small subset of cases (e.g., cases 26, 56, 15) show very low consistency, indicating that the model struggles with specific inputs.

These low-consistency cases represent difficult or ambiguous examples where the model's predictions are highly sensitive to training randomness. Small changes in patch order lead to different outcomes, indicating that these cases lie near the model's decision boundary.

### Split 1

- Case 56 → 10%
- Case 84 → 70%
- Case 92 → 60%

### Split 3

- Case 15 → 20%
- Case 46 → 20%
- Case 51 → 60%
- Case 61 → 50%
- Case 75 → 60%
- Case 93 → 70%

### Split 2

- Case 26 → 0%
- Case 32 → 40%
- Case 45 → 20%
- Case 86 → 50%
- Case 111 → 50%

### Split 4

- Case 94 → 20%

### Split 5

- Case 18 → 30%
- Case 47 → 50%
- Case 69 → 70%
- Case 96 → 50%

# Effect of patch order on accuracy:

Zoe

## Correlation of Val Loss and Accuracy

Relationship is not perfectly consistent across splits

Lower validation loss does not always mean higher accuracy

Split 4 achieves highest accuracy despite highest loss

Validation loss is a useful but imperfect proxy

### **Note: no per-seed correlation**

Because test accuracy was not saved per seed, we approximate accuracy using classification consistency, which still reflects overall performance trends.

| Split | Val Loss | Accuracy                |
|-------|----------|-------------------------|
| 1     | 0.1597   | 0.8778                  |
| 2     | 0.2851   | 0.8105                  |
| 3     | 0.2323   | 0.8316                  |
| 4     | 0.3400   | <b>0.9158 (highest)</b> |
| 5     | 0.2165   | 0.8722                  |

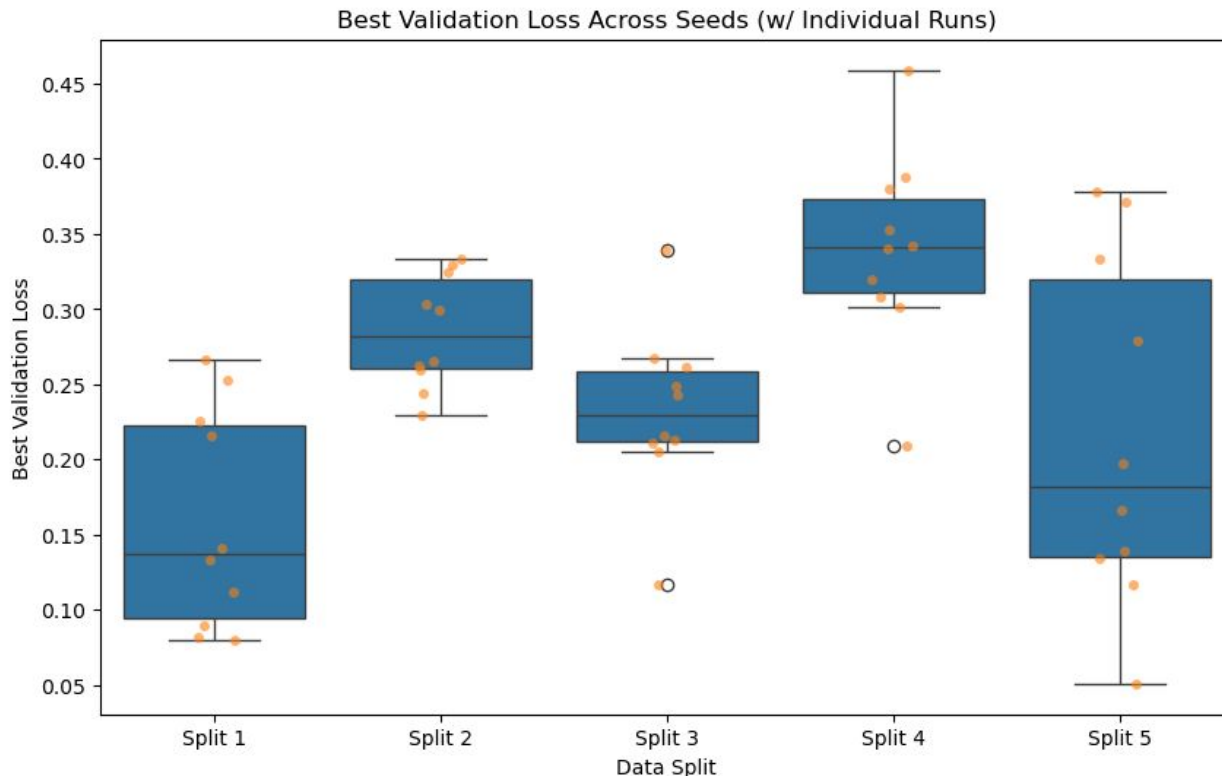
# Effect of Patch Order on Accuracy:

## Best Validation Loss

Isabella

GitHub:

[Presentation\\_4\\_code\\_Bella/best\\_validation\\_loss\\_code.ipynb](#)



**Best overall performance:**

**Split 1/Split 3:** lowest

median + tighter(ish)

variance;

model consistently performs well

**Worst overall performance:**

**Split 4:** highest median; some

runs exceed 0.4 = difficulty for the model

**Most unstable split:**

**Split 5:** wide box, extreme outliers; performance is inconsistent.

# Effect of Patch Order on Accuracy:

## Understanding Predicted Probability Graphs

Isabella

Looking at how predicted probabilities change across random seeds for the same test case:

- Model stability varies across test cases
  - a. Tight boxplots = consistent predictions across seeds
  - b. Wide boxplots = predictions vary a lot
    - i. Model not equally reliable for all inputs
- Some cases highly sensitive to randomness
  - a. Some boxplots span a large range (near 0 to near 1)
  - b. Model can give completely different predictions depending on initialization
- Confident predictions tend to be more stable
  - a. Cases with probabilities near 0 or 1 usually have tight boxes
  - b. Confident model also more consistent
- Mid-range probabilities = uncertainty
  - a. Cases centered around  $\sim 0.3$ – $0.7$  often show more spread
    - i. Ambiguous cases, model struggles

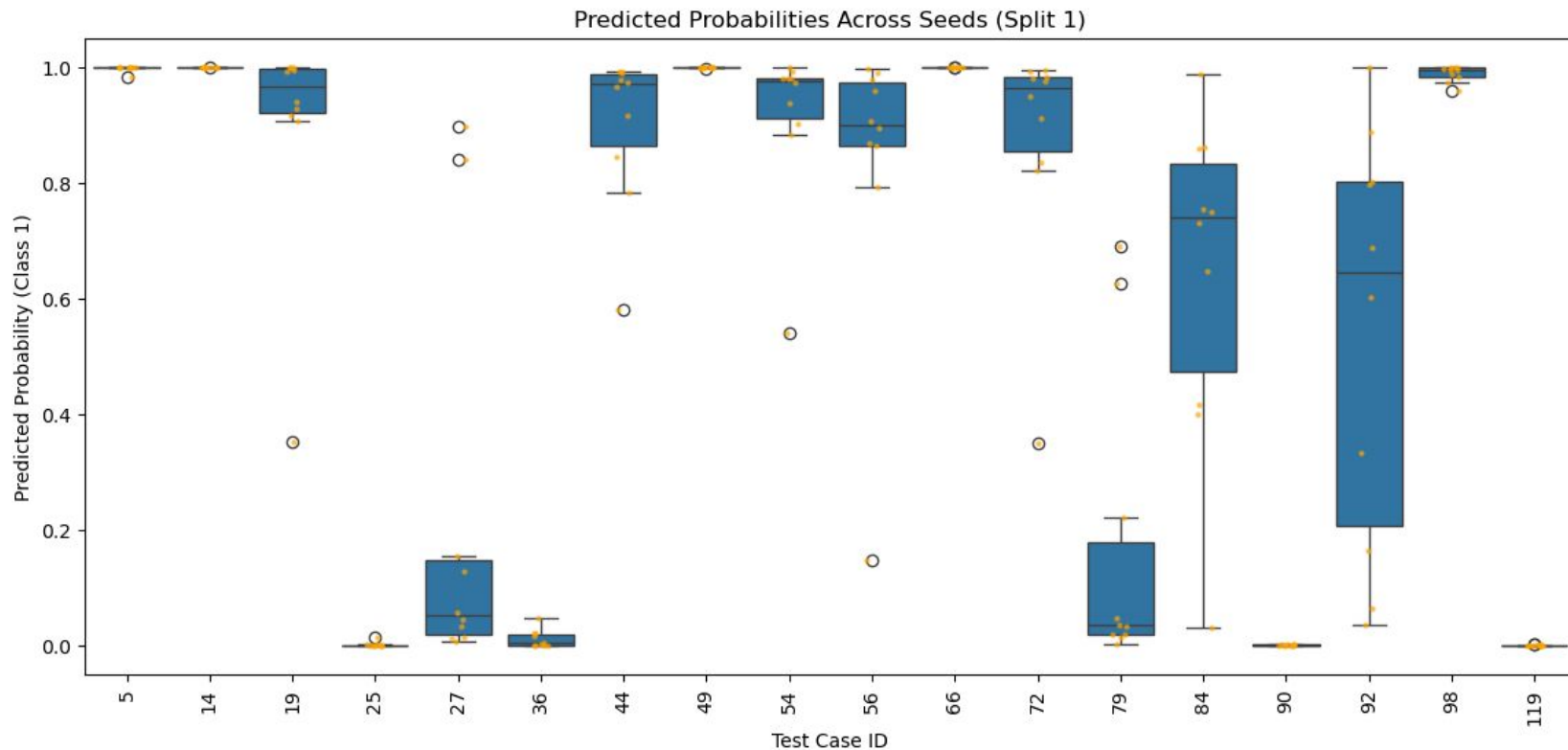
# Effect of Patch Order on Accuracy:

## Predicted Probabilities - SPLIT 1

Isabella

GitHub:

Presentation\_4\_code\_Bella/predicted\_probabilities\_code.ipynb



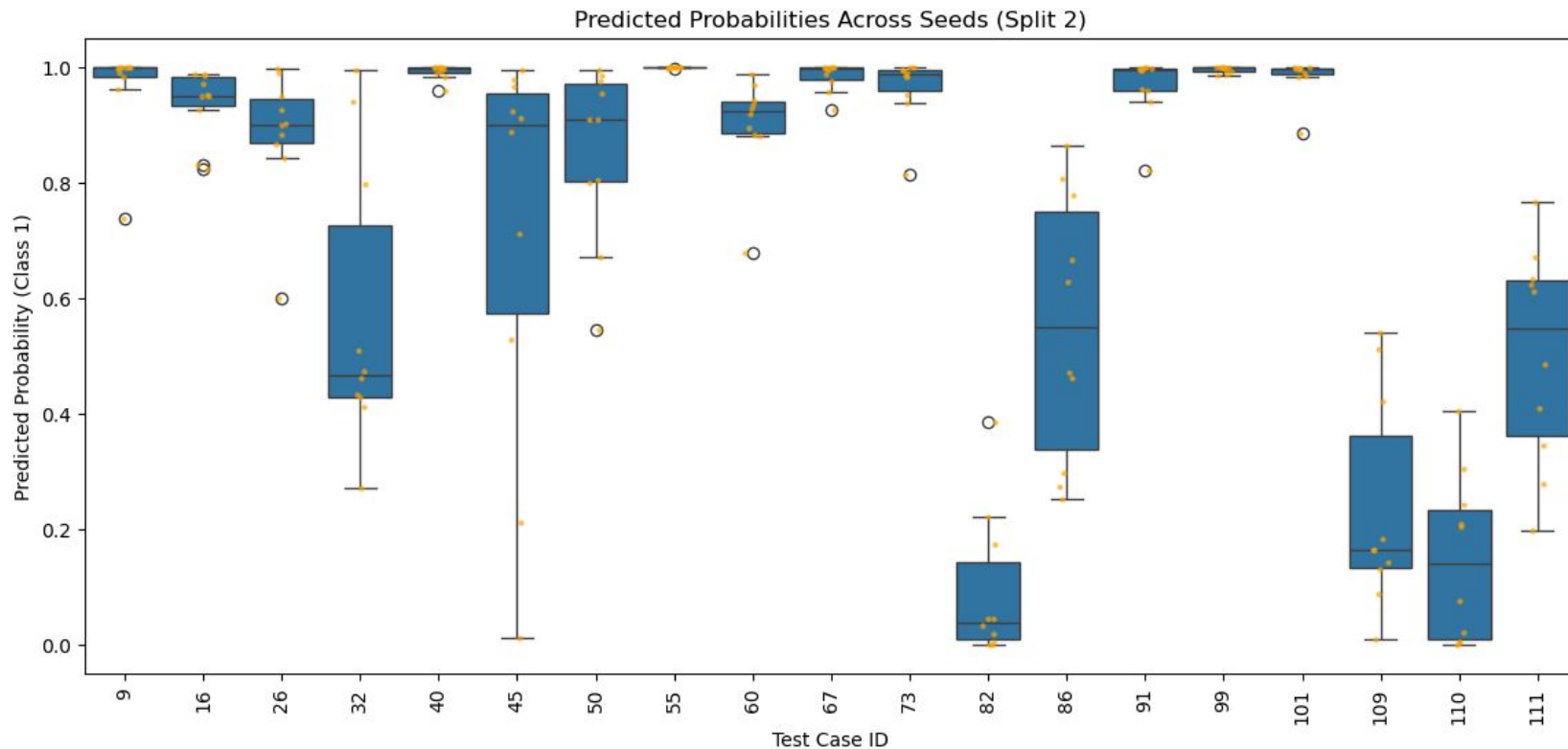
# Effect of Patch Order on Accuracy:

## Predicted Probabilities - SPLIT 2

Isabella

GitHub:

Presentation\_4\_code\_Bella/predicted\_probabilities\_code.ipynb



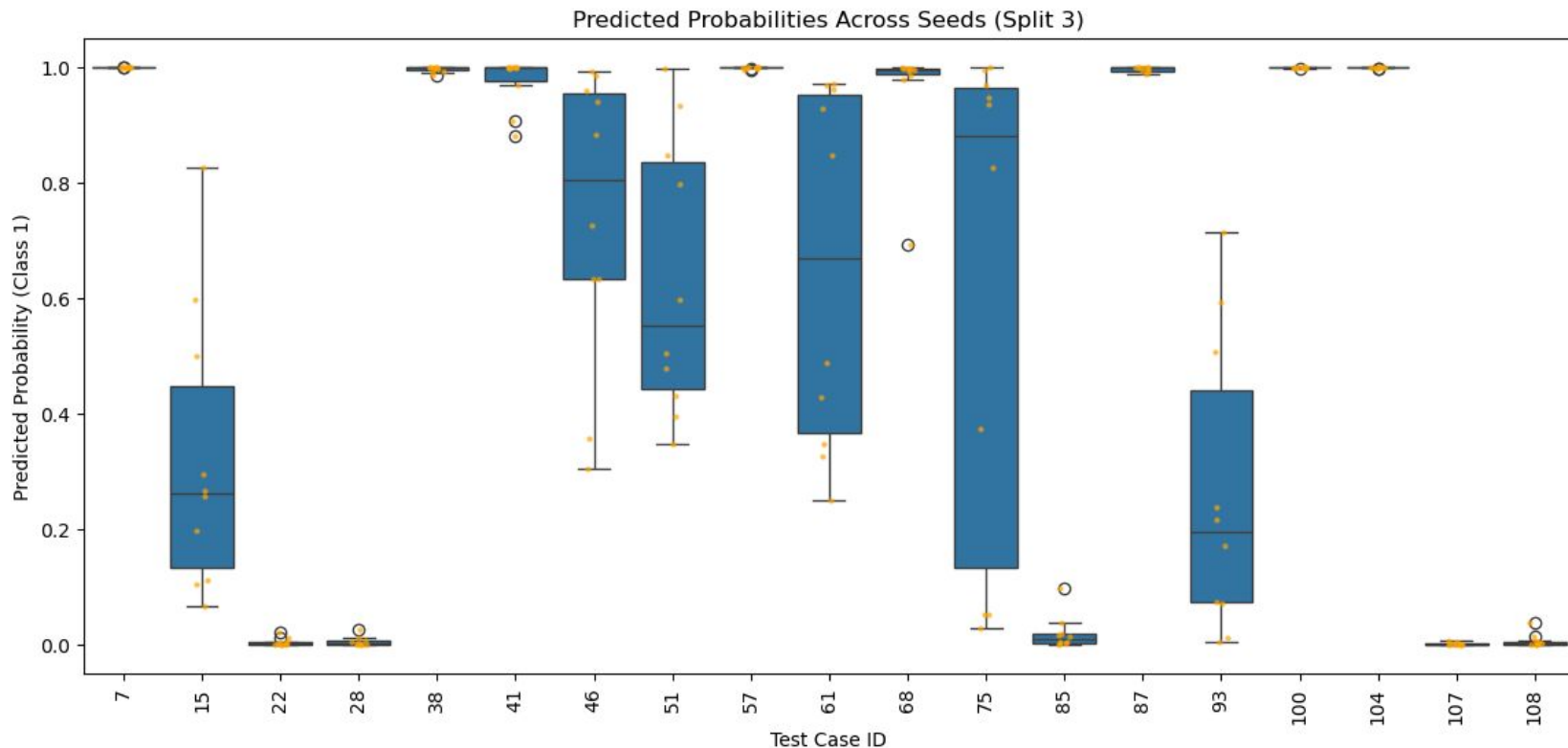
# Effect of Patch Order on Accuracy:

## Predicted Probabilities - SPLIT 3

Isabella

GitHub:

[Presentation\\_4\\_code\\_Bella/predicted\\_probabilities\\_code.ipynb](#)



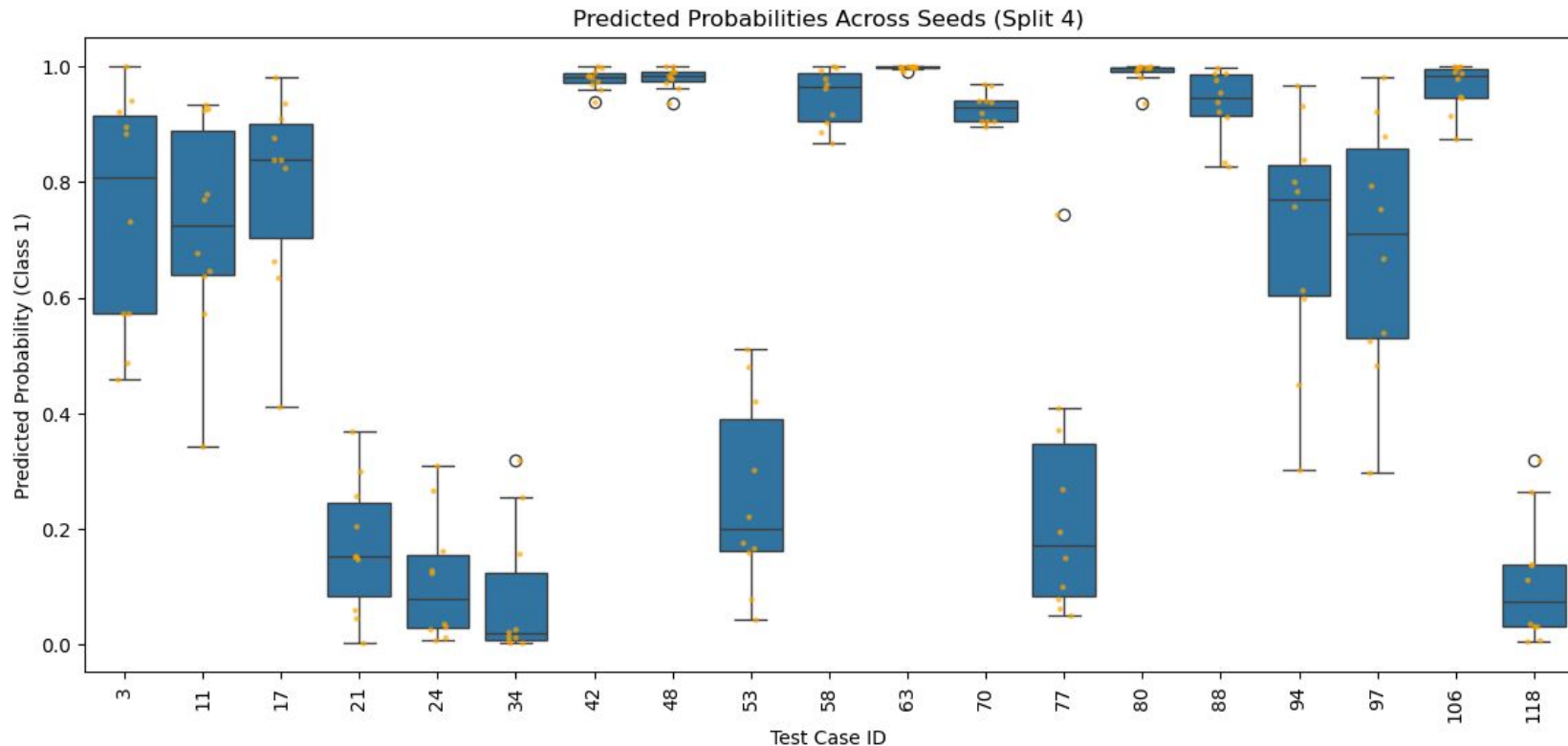
# Effect of Patch Order on Accuracy:

## Predicted Probabilities - SPLIT 4

Isabella

GitHub:

Presentation\_4\_code\_Bella/predicted\_probabilities\_code.ipynb



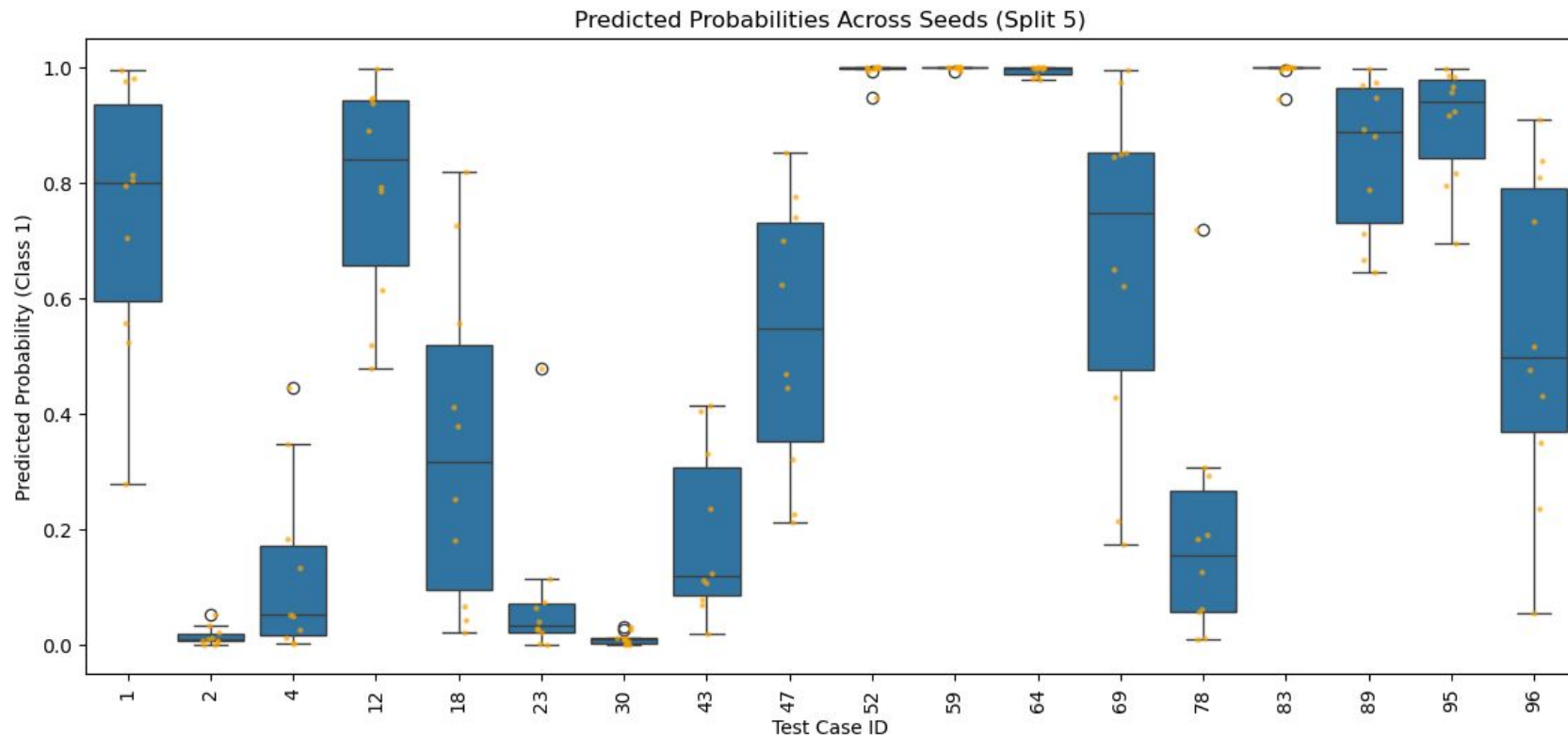
# Effect of Patch Order on Accuracy:

## Predicted Probabilities - SPLIT 5

Isabella

GitHub:

[Presentation\\_4\\_code\\_Bella/predicted\\_probabilities\\_code.ipynb](#)



# Coder Objective 2:

## Personal Reflection

Zoe

**Project Reflection: Please answer the following questions:**

- **Describe your key learnings from the course, including:**
  - **Technical learnings**
    - Visual studio code, QUEST, Github, Python in a real-world environment with real classified data, using AI as an assistant in coding, understanding the math and theory behind code and its changes
  - **Non-technical learnings (e.g., logistics, workflow, teamwork)**
    - More confidence in understanding coding and terminology, learning how to code in tandem with others, improved problem solving and communication skills
- **If you were to undertake a similar project again, what would you do differently?**
  - More intentional in incremental changes, knowing how to cut down run times earlier on by running for results we need and cutting for parts we don't
- **Which learnings from this course will you apply in a real-world data science project, whether in industry or research**
  - Importance of reproducibility, evaluating models on consistency and stability rather than just accuracy, communicating and understanding a project at different skill levels and being able to work together to meet each other where we're at → building on what everyone brings to the table

# Coder Objective 2 Reflection

## (part 1)

Zoe

01

### Why We Care About Randomness

Our model trains with a **batch size of 1**, which makes training **highly sensitive to data order**. The loss function is **non-convex** so there are many possible local optima.

So,

different random seeds → different training paths  
different random paths → different final models

We cannot trust a single run to represent performance.

02

### What We're Testing

We want to know if the model is stable across randomness? Do predictions stay consistent? How much does the model vary?

So, we run **5 different data splits** and use **10 random seeds per split**. This lets us separate variability from both **data splitting** and **training randomness**.

03

### Why These Metrics Matter

We use 4 key metrics:

1. **Best validation loss**
  - a. shows how training behavior varies across runs
2. **Predicted Probabilities**
  - a. captures how confident the model is for each case
3. **Classification Consistency**
  - a. measures how often the model makes the *same prediction* for a case
4. **Accuracy Variability (min/max)**
  - a. shows how much overall performance fluctuates

# Coder Objective 2 Reflection (2): Performance and Stability

Zoe

01

## Validation Loss

Validation loss varies across seeds, suggesting that training can converge to slightly different solutions. In most cases, lower validation loss is associated with improved model performance, but this relationship is not consistent across all splits.

02

## Predicted Probabilities

Predicted probabilities show some variation across runs, particularly for certain cases. This suggests that the model's confidence can depend on patch ordering, especially for less clear-cut examples.

03

## Classification Consistency

Most cases are classified correctly in the majority of runs, indicating relatively stable predictions. However, a small number of cases show lower consistency, suggesting that the model is less certain on those examples.

04

## Accuracy Variability

Test accuracy varies across seeds within each split, indicating some sensitivity to patch ordering. However, the overall range is not large, suggesting that performance is fairly stable.

05

## Final Reflection

Because batch size is 1, the order of patches affects the sequence of updates during training, which can lead to slightly different outcomes across runs. Overall, the model shows relatively stable performance, but some variability is present. This variability appears to be driven by a small number of less consistent cases, rather than widespread instability across the dataset.